Diseño del Sistema de Adquisición y Visualización Multiplexada de Temperatura y Voltaje.

En este documento se detalla la implementación de un diseño RTL para el control del bloque IP XADC, con la finalidad de medir temperatura y voltajes internos del circuito integrado. Además, se describen los módulos necesarios para la correcta adecuación de la salida del ADC para su posterior visualización decimal en el visualizador 7 segmentos. También se explican las conexiones entre la unidad de control, el XADC y las entidades de la etapa de acondicionamiento.

1. Especificaciones y Descripción Funcional del Diseño.

El principal objetivo de este diseño es muestrear la temperatura y el voltaje de la FPGA haciendo uso del ADC interno disponible en la Artix-7 XC7A100T, denominado XADC [26]. Las magnitudes físicas seleccionadas han sido la temperatura interna, el voltaje de alimentación y el voltaje de la VRAM.

En primer lugar, para seleccionar cual magnitud será mostrada en el visualizador, se contará con tres entradas externas conectadas a interruptores de la placa NEXYS 4 DDR. Se ha usado el SW1 para la temperatura, el SW2 para indicar que se va a medir voltaje y el SW3 para discernir entre voltaje de alimentación o de VRAM. Dado que no se puede controlar el correcto uso de los interruptores por parte del usuario, se define un orden de prioridad en los pulsadores mostrado en la Tabla 1.

Tabla 1. Orden de prioridad en la visualización de magnitudes físicas.

Interruptores			Magnitud Visualizada		
SW1	SW2	SW3	Temperatura	Vcc	VRAM
0	0	X	No	No	No
0	1	0	No	Sí	No
0	1	1	No	No	Sí
1	X	X	Sí	No	No

De los datos de la Tabla 6.3 se puede observar que la máxima prioridad reside en la temperatura, es decir, si SW1 está activo en alto se ignorará el valor del resto de interruptores, muestreando y visualizando la temperatura. En el caso donde SW1 y SW2 están a cero, no se mostrará ninguna magnitud en el visualizador, mostrando el dígito cero en cada uno de los 7 segmentos.

En segundo lugar, el formato de representación numérica de cada una de las magnitudes en el visualizador 7 segmentos será la siguiente:

- Se emplearán 4 dígitos de 7 segmentos para representar el valor de cada magnitud.
- El formato de representación para la temperatura estará en grados centígrados, usando los tres dígitos superiores para representar la parte entera, desde las unidades hasta las centenas, el último dígito mostrará la parte decimal (Fig.1).

- El formato de representación para los voltajes estará en unidades de Voltios, empleando el primer dígito para la parte entera y el resto para la parte decimal (Fig.2).
- Cuando el valor de la magnitud no necesite emplear todos los dígitos para ser representada, en los dígitos nulos se visualizará un cero (Fig.1).

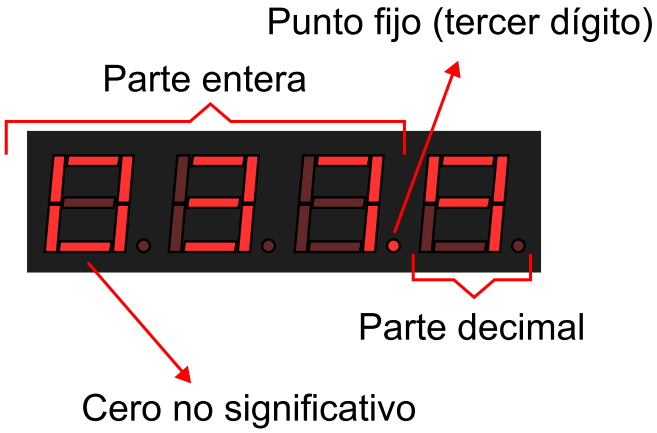


Fig.1. Ejemplo de formato de representación numérica para una temperatura de 37.9 °C en el visualizador. Se emplea notación de punto fijo con tres dígitos para la parte entera y uno para la parte decimal. Los dígitos no significativos se representan con ceros, es decir se hace un relleno de ceros a la izquierda.

Fig.2. Ejemplo de formato de representación numérica para voltaje de 1.002 V en el visualizador. Se emplea notación de punto fijo con un dígito para la parte entera y tres para la parte decimal.

Tras la descripción funcional externa del sistema pasamos a centrarnos en la descripción funcional interna. El bloque IP XADC tiene como salida los datos muestreados con una resolución de 12 bits en formato binario natural o en Ca2, dependiendo de la magnitud que se esté muestreando. Debido a que estos valores no se corresponden directamente al valor binario real de las magnitudes físicas, se ha implementado una etapa de acondicionamiento de señal digital como se muestra en la Fig.3.

La arquitectura del sistema se compone de las siguientes etapas o entidades VHDL:

1. Funciones de Transferencia: se han implementado dos bloques aritméticos encargados de convertir el valor crudo de 12 bits a la salida del XADC a unidades

físicas. El primero de ellos realiza la conversión de la temperatura a grados centígrados empleando la operación aritmética correspondiente (`func_trans.vhd`). El segundo calcula el valor del voltaje en milivoltios aplicando la operación aritmética para el voltaje (`func_trans_volt.vhd`).

2. Cálculo del Valor Absoluto para Temperatura: dado que el rango de medida de la temperatura presenta parte negativa, el valor de salida del bloque de función de transferencia puede ser un dato negativo representado en Ca2. Si un valor negativo se mostrará en el visualizador sin un tratamiento previo, se interpretaría como un valor positivo muy elevado, causando el funcionamiento incorrecto del sistema. Para paliar este inconveniente, se ha añadido una etapa a la salida de la función de transferencia de la temperatura (`valor_abs.vhd`) que calcula el valor absoluto del dato de entrada, indicando con una señal si el signo es positivo o negativo (`signo_o`).

3. Conversión a Formato BCD: una vez obtenidas las magnitudes físicas en binario natural positivo, la entidad `bin2bcd.vhd` transformará el dato a formato Decimal Codificado en Binario (BCD). De esta forma, se obtienen los diferentes órdenes del dato decimal (unidades, decenas, centenas, etc.) separados por dígitos, contando cada uno de ellos con cuatro bits.

4. Control de la Visualización en el *Display*: con el dato en formato BCD, la entidad `visu7seg.vhd` se encargará de traducir dicho valor a las entradas de control del visualizador 7 segmentos para representarlo correctamente. Para ello traduce el formato BCD a 7 segmentos y controlará el encendido y apagado de los ánodos y cátodos de los segmentos.

5. Unidad de Control: por último, todos los bloques anteriores deberán ser controlados según la combinación de las entradas seleccionadas por los interruptores SW1, SW2 y SW3 (`temp_sel`, `volt_sel`, `vram_sel`). Para ello se diseña un proceso secuencial (`Process P1`) dentro de la entidad `adc_top_level.vhd`, que gestionará la selección de la magnitud a muestrear por el XADC y cómo se enruta dicho valor a los distintos bloques según la magnitud seleccionada.

Todos los bloques previamente definidos se muestran conectados entre sí en el diagrama de bloques de la Fig.3. Aunque se hayan obviado las señales de reloj y *reset*, todos los bloques, incluido el proceso de control secuencial, son síncronos con señal de *reset* asíncrona activa en alto. En los siguientes subapartados se procederá a definir en profundidad las entidades VHDL y el proceso de control aquí nombrados.

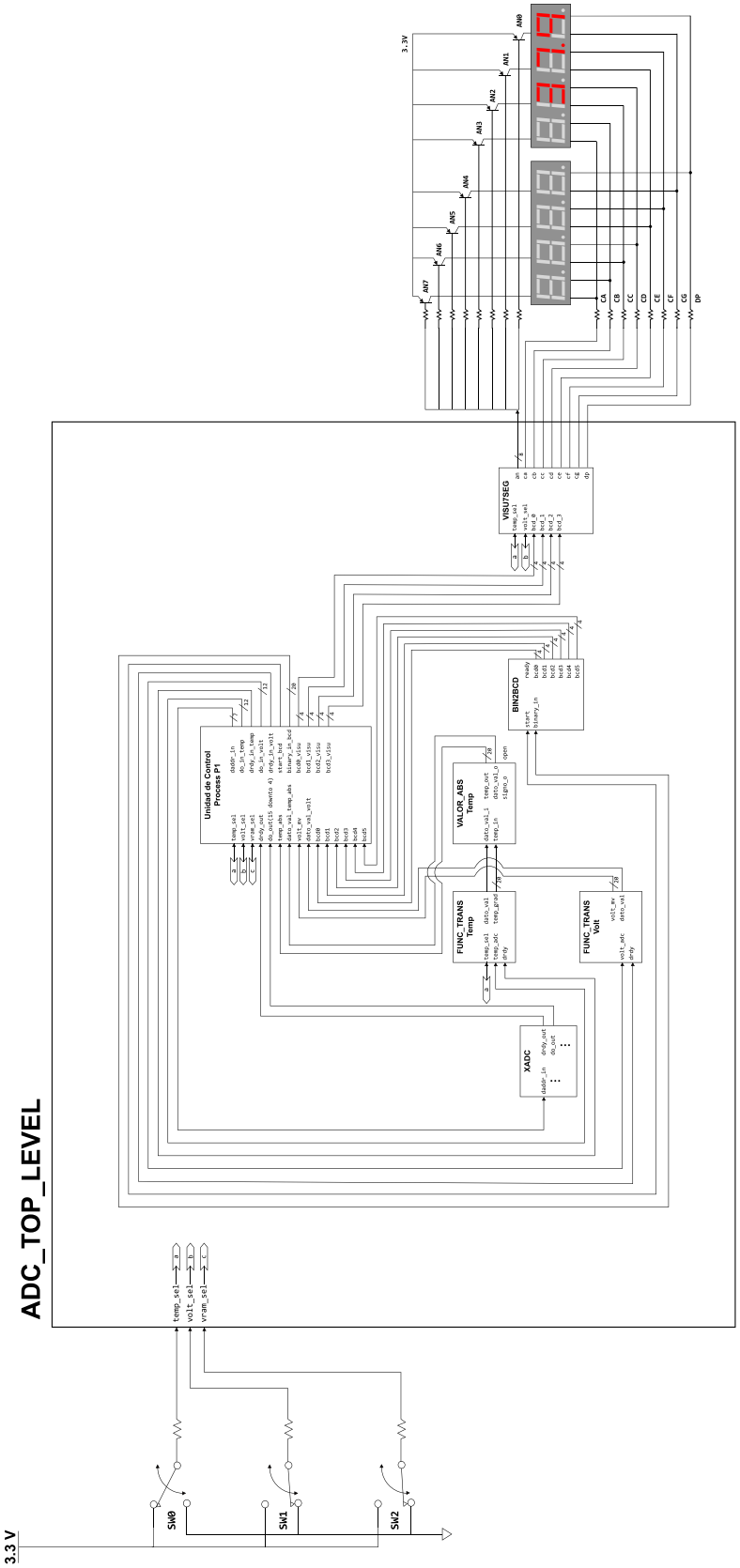


Fig.3. Arquitectura RTL del sistema de adquisición, acondicionamiento y visualización de las magnitudes internas del dispositivo Artix-7 XC7A100T.

2. Bloque IP XADC.

El XADC es un convertidor analógico a digital (ADC) disponible internamente en las FPGAs de la serie 7 de Xilinx. Para hacer uso del XADC se ha empleado el *IP Catalog* de la herramienta Vivado.

Entre las principales características del XADC destaca la posibilidad de muestrear señales analógicas tanto externas como internas. Las variables internas son accesibles gracias a sensores integrados en la propia FPGA y conectados al propio convertidor (*On-Chip Sensors*), siendo esta última opción la empleada en el diseño.

La resolución del convertidor es de 12 bits en un rango de 1V, aunque los datos adquiridos se almacenan en registros de 16 bits, donde solo los 12 bits más significativos representan el valor de la magnitud muestreada, Fig.4.

DI15	DI14	DI13	DI12	DI11	DI10	DI9	DI8	DI7	DI6	DI5	DI4	DI3	DI2	DI1	DI0
DATA [11:0]												Note			

Fig.4. Estructura de los registros de estado del XADC, extraído de [26].

Para conectar el XADC con la FPGA se emplea una interfaz de registros. Dicha interfaz cuenta con un banco de registros de estado (*status registers*), que contienen el valor de los datos muestreados, y un banco de registros de control (*control registers*), que contiene parámetros de configuración, Fig.5. Para acceder a dichos registros se emplea el *Dynamic Reconfiguration Port* (DRP).

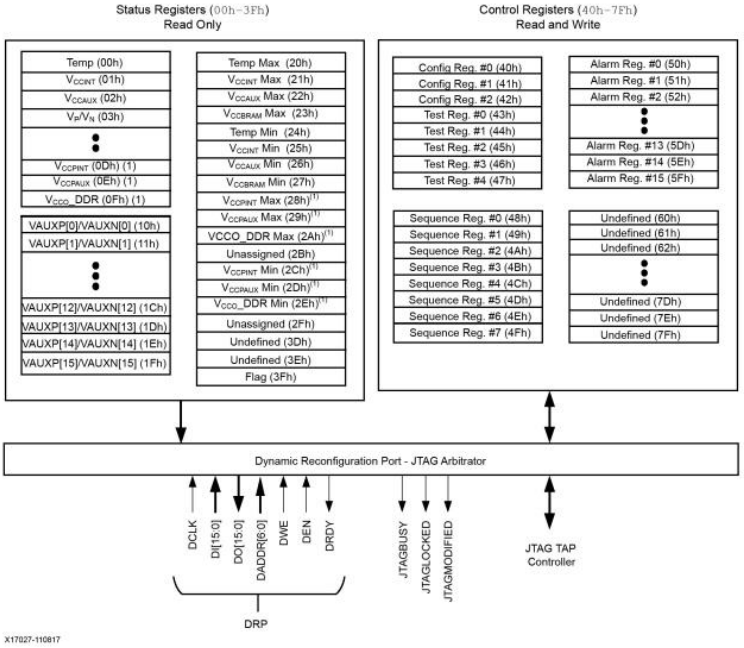


Fig.5. Banco de registros de estado y control del XADC, extraído de [26].

Dado que los valores de los registros de control se preestablecen usando el *IP Catalog*, desde el punto de vista del sistema, solo tienen relevancia los registros de estado para acceder al contenido de las variables adquiridas. Este banco es de solo lectura, luego para acceder al contenido de sus registros, simplemente se debe indicar la dirección del registro (`DADDR[6:0]`) y el XADC pondrá el valor en la salida de datos del DRP (`DO[15:0]`). Cada vez que exista una nueva lectura disponible se indicará por la salida `DRDY` del DRP.

Teniendo en cuenta lo anterior, se puede simplificar la interfaz de entrada/salida del bloque XADC a la mostrada en la Fig.6. Esta simplificación es totalmente válida a nivel funcional para el sistema personalizado en cuestión. A pesar de esto, el resto de las entradas y salidas no desaparecen.

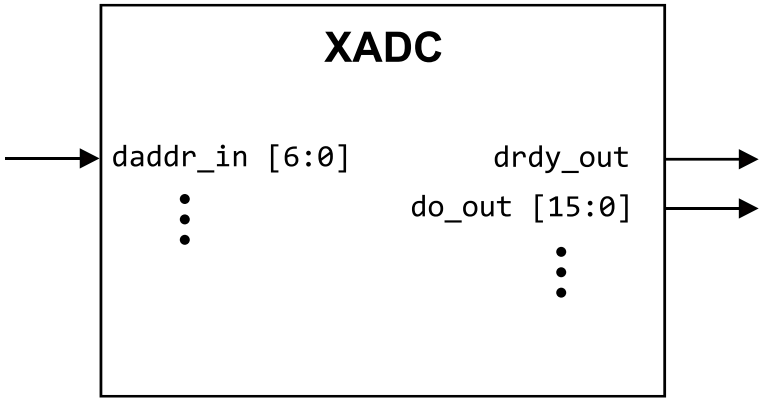


Fig.6. Interfaz de entrada/salida simplificada del bloque XADC.

Los registros de estado de cada una de las magnitudes seleccionadas para este trabajo y sus direcciones de memoria se muestran en la Tabla 2. Para comprender mejor el funcionamiento de las señales del convertidor se describe un diagrama temporal a modo de ejemplo en la Fig.7.

Tabla 2. Registros de estado de las magnitudes del diseño.

Magnitud	Nombre del Registro	Dirección	Descripción
Temperatura Interna	Temperature	0x00	Contiene el resultado del valor medido del sensor de temperatura interno en sus 12 MSBs.
Voltaje Alimentación Interno	VCCINT	0x01	Contiene el valor medido por el sensor de alimentación interna (VCCINT) en sus 12 MSBs.
Voltaje BRAM Interno	VCCBRAM	0x06	Contiene el valor medido por el sensor de alimentación de la memoria BRAM (VCCBRAM) en sus 12 MSBs.

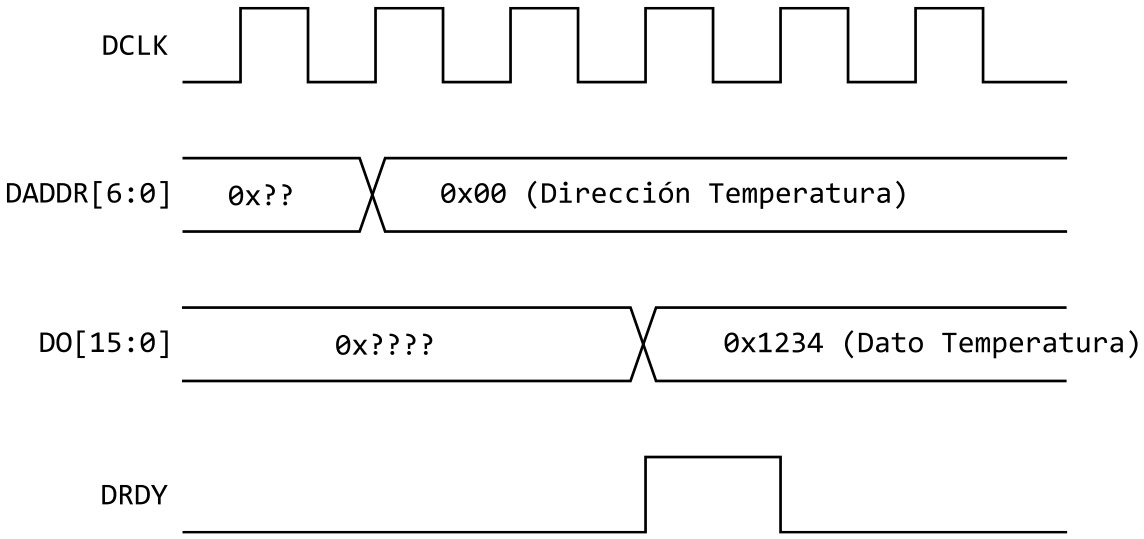


Fig.7-7. Diagrama de temporización de las señales del *Dynamic Reconfiguration Port* (*DRP*) relevantes en el sistema de visualización.

Como puede observarse, la señal `DRDY` debe emplearse como un indicador para capturar el contenido del registro cuya dirección se haya establecido en `DADDR[6:0]`, es decir, durante el ciclo de reloj completo que `DRDY` esté activo en alto, el dato del registro (`DO[15:0]`) de estado es válido.

Además, cabe mencionar el modo de operación que se ha configurado en el bloque a través del IP Catalog: el modo *Continuous Sequence* (Secuencia Continua). Al seleccionar esta opción, el XADC se encarga de muestrear y convertir cíclicamente las variables habilitadas (Temperatura, VCCINT y VCCBRAM) de forma automática, sin necesitar una señal de inicio externa. De este modo, los registros de estado se actualizan constantemente en segundo plano con el último valor convertido. Esto facilita el diseño del sistema, ya que la lectura a través del puerto DRP no necesita sincronizarse con el proceso de conversión. Simplemente se accede al registro deseado sabiendo que el dato almacenado corresponde siempre a la medición más reciente.

Una vez comprendido esto, el control externo del bloque XADC se puede resumir en el diagrama de flujo de la Fig.8. Dependiendo del valor de los selectores externos (SW1, SW2 y SW3), la dirección del bus DRP se cambiará para acceder a un registro de estado u otro. El valor de la señal `DRDY` se propagará al resto de bloques para indicar que el dato disponible en `DO[15:0]` es válido y puede comenzar el acondicionamiento de los datos crudos.

Dado que la función de transferencia y el formato de representación numérico para las variables VCCINT y VCCBRAM son iguales, se empleará el interruptor SW2 para enrutar todas las señales hacia los bloques de acondicionamiento para voltaje, lo cual denominamos modo voltaje. Esto permite emplear SW1 y SW2 para configurar la etapa de acondicionamiento para temperatura o voltaje, respectivamente. En cambio, SW3 permitirá discernir entre las dos variables de tipo voltaje, eliminando así la posibilidad de conflictos en la selección entre ambos, evitando añadir otra prioridad al diseño.

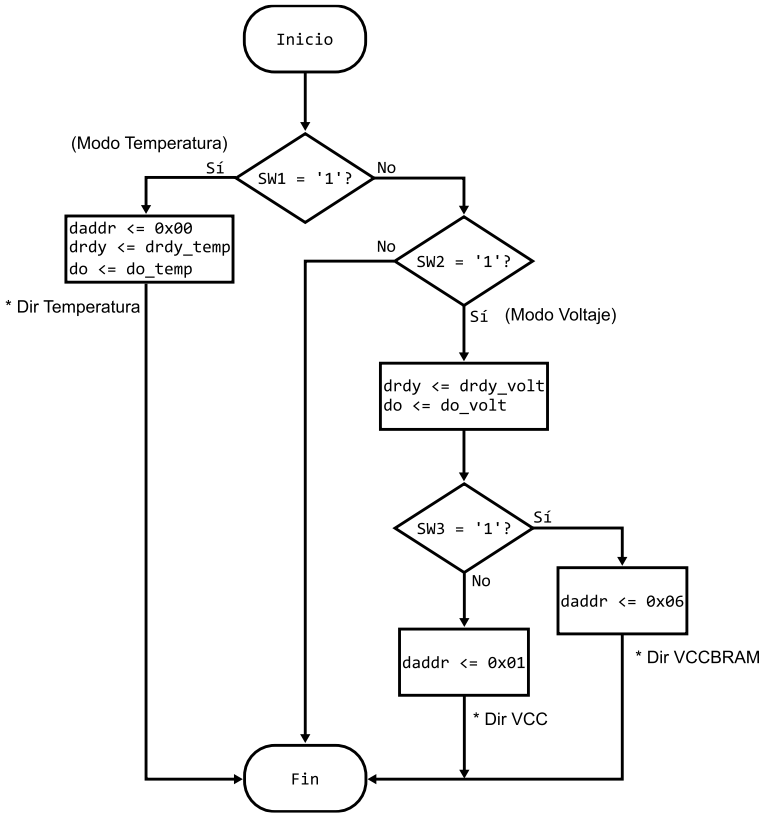


Fig.8. Diagrama de flujo con el árbol de decisiones para el control y enrutado de las señales del XADC.

3. Bloque para la Función de Transferencia de la Temperatura.

En el caso de que se haya seleccionado la temperatura interna, para calcular el valor medido en grados centígrados es necesario aplicar la siguiente operación aritmética, que es la que se muestra en la sección *Analog Inputs* de [26]:

$$\text{Temperatura}(°\text{C}) = \frac{\text{Valor ADC} \times 503.975}{4096} - 273.15 \tag{1}$$

Como se observa en la ecuación (1), la función de transferencia presenta decimales. Implementar en hardware la ecuación de forma literal supondría el coste añadido de emplear aritmética de punto flotante, con el correspondiente gasto en recursos. Otra posible solución, adoptada en este trabajo, se basa en escalar la función por un factor de 1000 en ambos miembros.

Esta técnica consiste en operar con miligrados en lugar de grados centígrados para así realizar todos los cálculos con aritmética de enteros, fácilmente implementable gracias a los bloques DSP disponibles entre los recursos de la FPGA. De este modo, la ecuación original se puede reescribir como en (2).

$$\text{Temperatura}(\text{m}°\text{C}) = \frac{\text{Valor ADC} \times 503975}{4096} - 273150 \tag{2}$$

Analizando la expresión, ahora la multiplicación decimal se convierte en una multiplicación entera. Además, la división, al ser potencia de dos, se puede implementar como 12 desplazamientos binarios a la derecha, evitando la implementación de algoritmos de división binaria.

Otra forma de calcular la expresión algebraica es dividiendo 503975 entre 4096, lo cual se puede aproximar por la constante 123 cometiendo un error de 0.167°C, valor despreciable si se tiene en cuenta la tolerancia de ±4°C del sensor interno. Con esta simplificación añadida se consigue reducir el tamaño de almacenamiento de la constante de multiplicación, (3).

$$\text{Temperatura(m°C)} = \text{Valor ADC} \times 123 - 273150 \tag{3}$$

Por otro lado, dado que el rango del ADC va desde cero hasta 4095, el rango de la temperatura escalada oscilará entre los -273150 a los 230535 m°C. Para poder almacenar dicho rango de valores en un registro se necesitarán 20 bits, `temp_grad[19:0]`.

Aunque solo sean necesarios 20 bits, para la operación interna (`res_int`) se ha optado por un ancho de 21 bits para garantizar un bit de seguridad, evitando la posibilidad de desbordamiento en la operación de resta.

Además, para materializar la ecuación en el hardware se han definido dos constantes enteras (`signed`). La primera, `mul_123`, contiene el entero 123. La segunda constante, `kelvin_0`, almacena el término correspondiente al cero absoluto en Kelvin (-273150). De esta forma la implementación de la función de transferencia se traduce a la expresión (4).

$$\text{res_int[20:0]} = \text{temp_adc[11:0]} \times \text{mul_123[7:0]} + \text{kelvin_0[20:0]} \tag{4}$$

Una vez definida la operación matemática interna del bloque, su interfaz de entrada/salida se define en la Fig.9. Por otra parte, la implementación del control del bloque se ha definido mediante una máquina de estados finitas representada por la carta ASM de la Fig.10. La FSM presenta los siguientes estados:

1. Espera: el bloque permanece inactivo hasta que el bloque XADC indica con su salida `drdy = '1'` que hay un nuevo dato disponible para procesar y, además, el selector de modo selecciona temperatura (`temp_sel = '1'`).
2. Cálculo: en este estado se lleva a cabo la operación descrita en la ecuación (4).
3. Fin: se actualiza el valor del registro de salida `temp_grad` con el dato calculado en el estado anterior y se genera un pulso en la señal `dato_val` durante un ciclo de reloj para indicar al siguiente bloque, en este caso `VALOR_ABS`, que hay un nuevo dato para procesar.

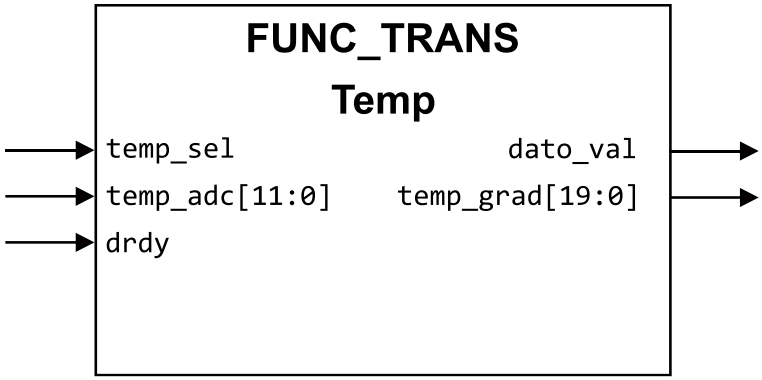


Fig.9. Interfaz de entrada/salida del bloque funcional `FUNC_TRANS` para la temperatura. Se han omitido las señales de reloj y *reset* por claridad.

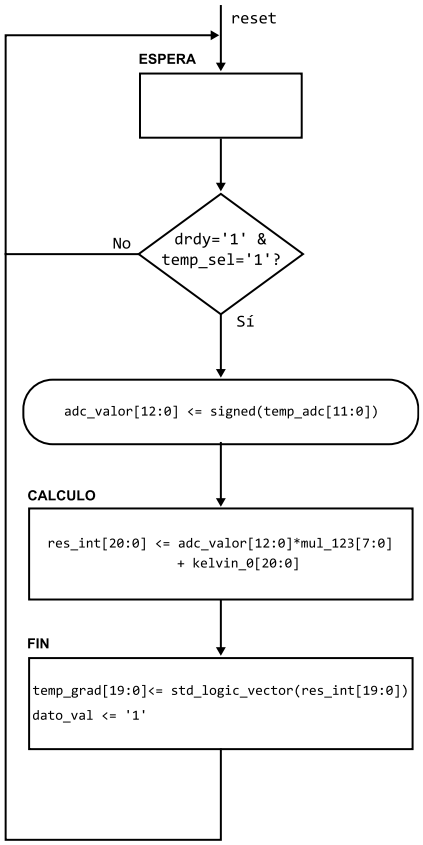


Fig.10. Carta ASM del proceso de control para el cálculo de la función de transferencia de la temperatura.

Cabe mencionar que el diseño VHDL de esta entidad correspondiente al bloque de cálculo de la función de transferencia de la temperatura, se encuentra de forma íntegra en la dirección `src/func_trans.vhd`.

Finalmente, desde el punto de vista de la unidad de control (*Process P1*), cuando el modo temperatura está seleccionado (`temp_sel = '1'`), la unidad de control se encarga de

conectar las salidas del bloque XADC al bloque para la función de transferencia de la temperatura. Para conectar la salida del XADC `drdy` con la entrada de igual nombre del bloque de la temperatura se emplea la señal intermedia `drdy_in_temp`. De igual modo, para conectar `do_out[15:3]` con `temp_adc[11:0]` se emplea la señal `do_in_temp[11:0]`. La Fig.11 muestra gráficamente las conexiones mencionadas.

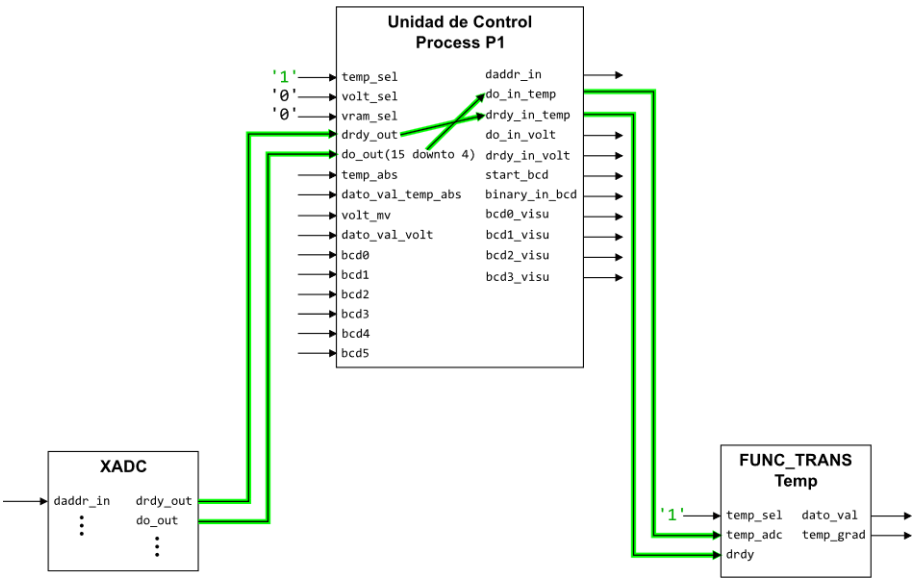


Fig.11. Diagrama de conexiones entre el bloque XADC y FUNC_TRANS para el modo temperatura vía la Unidad de Control (*Process P1*).

4. Bloque para la Función de Transferencia del Voltaje.

Cuando el modo seleccionado corresponda a visualización de voltaje, ya sea de alimentación o BRAM, la operación aritmética para convertir el dato binario crudo del XADC a unidades de voltios viene dado por (5), extraída de la sección *Power Supply Sensor* de [26] es:

$$\text{Voltaje } (V) = \frac{\text{Valor ADC}}{4096} \times 3V \tag{5}$$

De igual forma que el caso de la temperatura, para evitar implementaciones de aritmética en punto flotante se vuelve a emplear la opción de multiplicar en ambos miembros de la ecuación por 1000. De este modo, la función de transferencia se reescribe en unidades de milivoltios según (6). Al truncar la operación a enteros el error cometido es inferior a 1 mV en el peor de los casos.

$$\text{Voltaje (mV)} = \frac{\text{Valor ADC}}{4096} \times 3000 \text{ mV} \tag{6}$$

La constante de multiplicación se almacenará en `mul_3000`, con un ancho necesario de 12 bits para almacenar dicho valor. Dado que el dato del XADC, en el caso del voltaje, tiene un rango positivo, no es necesario extender su signo y, por tanto, `adc_valor` presenta un ancho de 12 bits. Debido a esto, la multiplicación podrá generar como máximo un dato de 24 bits de ancho, almacenado en la señal intermedia `res_int`. Para la división se realizará un desplazamiento lógico a la derecha (`slr`) igual a 12 posiciones. Esto último es en VHDL análogo a almacenar los 12 MSBs, descartando los restantes LSBs.

Por otra parte, aunque el rango de salida en mV se pueda representar en 12 bits, dado que el valor de salida de la temperatura necesita 20 bits, para reutilizar el mismo bloque `BIN2BCD` en ambos casos, se extenderán el dato de salida de voltaje a 20 bits. Por este motivo, será necesario separar el cálculo aritmético en las operaciones expresadas en (7) y (8).

$$res_int[23:0] = adc_valor[11:0] \times mul_3000[11:0] \tag{7}$$

$$volt_mv\ [19:0] = X"00"\ \&\ res_int[23:12] \tag{8}$$

La interfaz de entrada/salida del bloque se muestra a modo de diagrama de bloques en la Fig.12. Además, el control interno de la ejecución del bloque se ha definido como una máquina de estados finita representada de forma gráfica en la carta ASM de la Fig.13. Los estados son los siguientes:

1. Espera: mientras que `drdy` y `volt_sel` sean distintos de `'1'`, la FSM permanecerá en este estado. Cuando haya un nuevo dato disponibles por parte del XADC (`drdy='1'`) y el modo de selección corresponda con voltaje (`volt_sel='1'`), el dato de entrada (`volt_adc`) se convertirá a tipo `unsigned` para operar aritméticamente con él y se evolucionará al siguiente estado.

2. Cálculo: aquí se aplicará la primera parte de la operación aritmética de la función de transferencia correspondiente con la expresión (7) y se evolucionará al siguiente estado de forma incondicional.

3. Fin: por último, se aplicará la parte final de la operación aritmética correspondiente a (8). Además, se producirá un pulso de un ciclo de reloj en la señal `dato_val`, indicando a los bloques posteriores a este que hay un nuevo dato disponible para ser tratado.

Esta entidad, correspondiente al bloque de cálculo de la función de transferencia del voltaje, se encuentra de forma íntegra en la dirección `src/func_trans_volt.vhd`.

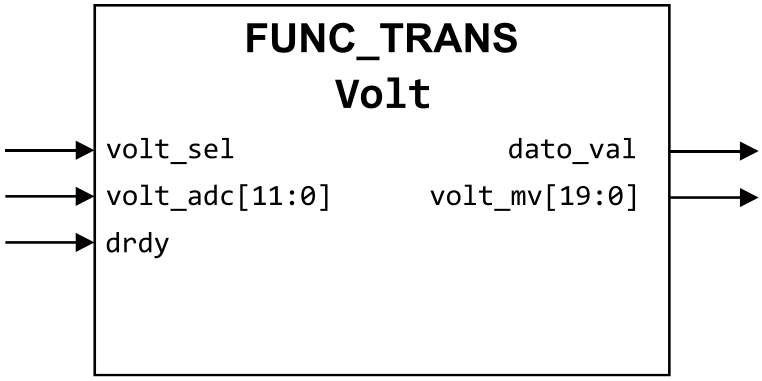


Fig.12. Interfaz de entrada/salida del bloque para el cálculo de la función de transferencia para el voltaje. Se han omitido las señales de reloj y *reset* por claridad.

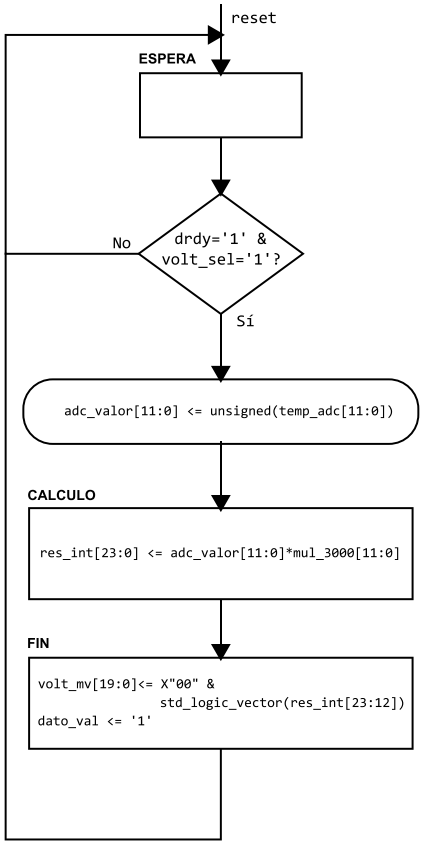


Fig.13. Carta ASM de la máquina de estados finita para el control del bloque de función de transferencia para el voltaje.

Para finalizar, desde el punto de vista de la Unidad de Control (*Process P1*), las conexiones entre las salidas del XADC y las entradas del bloque de función de transferencia para voltaje cuando el modo voltaje está activo se muestran en la Fig.14. La salida `drdy` se conecta con la entrada `drdy` a través de la señal intermedia `drdy_in_volt`. Por otra parte, la salida `do_out[15:3]` se conecta con la entrada `volt_adc[11:0]` mediante la señal `do_in_volt[11:0]`.

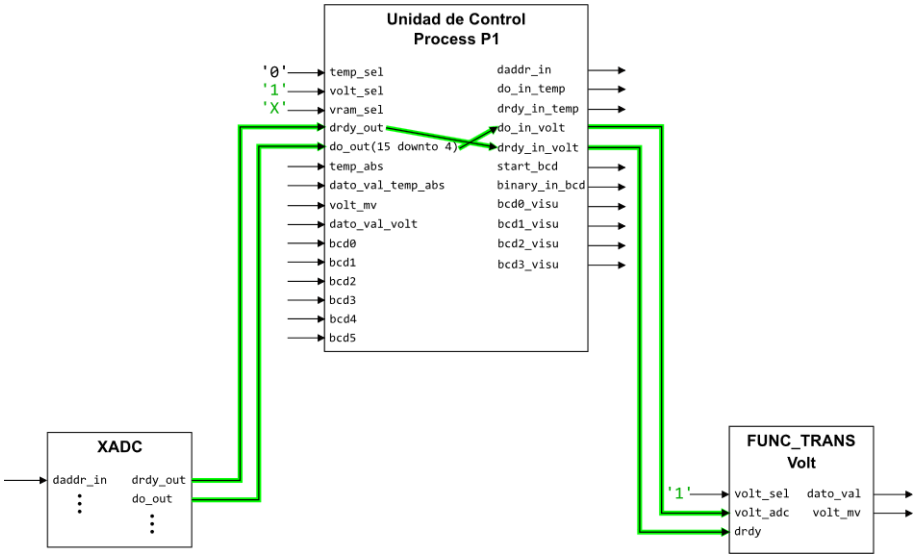


Fig.14. Diagrama de conexiones entre el bloque XADC y el bloque de función de transferencia para voltaje vía Unidad de Control (*Process P1*).

5. Bloque para el Cálculo del Valor Absoluto de la Temperatura.

En el caso del acondicionamiento de los datos de temperatura, en el paso previo a la conversión de binario natural a formato BDC, se necesita añadir un bloque adicional para realizar el valor absoluto sobre el dato binario de la temperatura en miligrados.

La causa de esta necesidad se explica por la existencia de valores negativos dentro del rango de operación del sensor de temperatura. Al tener signo, los datos de temperatura se representan en Ca2. Si un valor de temperatura negativo en Ca2 llegará sin más al bloque BIN2BCD, el algoritmo convertiría el valor negativo en un valor positivo muy elevado.

Para evitar el funcionamiento erróneo del sistema en rangos negativos de temperatura se ha realizado un bloque cuya funcionalidad se limite a detectar si el dato es negativo o no, y en caso afirmativo, calcular su valor absoluto, es decir, deshacer el complemento a dos. De esta forma, se devolverá el dato binario positivo y, para indicar el signo, se usará una señal de salida, evitando así una posible conversión errónea.

Para detectar el signo del valor binario de entrada se empleará la lectura del bit más significativo (bit de signo). Si el MSB es igual a `'1'` el valor será negativo y deberá calcularse su valor absoluto. En VHDL se puede realizar dicho cálculo operando con el tipo `signed` multiplicando por -1, (9).

$$temp_out[19:0] = std_logic_vector(-signed(temp_in)) \qquad (9)$$

En la Fig.15 se muestra el diagrama de la interfaz de entrada/salida de la entidad. Dada la sencillez de la operación efectuada por este bloque, el control de su ejecución no se ha realizado mediante máquina de estados finitos, en cambio, se empleará un diagrama de flujo con árboles de decisión para modelar el control del bloque, Fig.16.

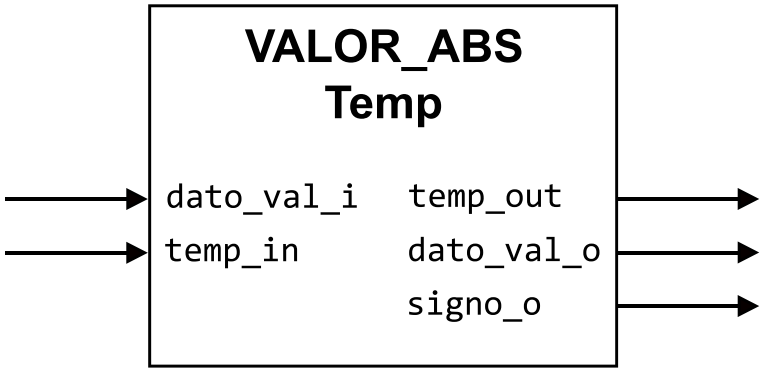


Fig.15. Interfaz de entrada/salida del bloque para el cálculo del valor absoluto de la temperatura. Se han omitido las señales de reloj y *reset* por claridad.

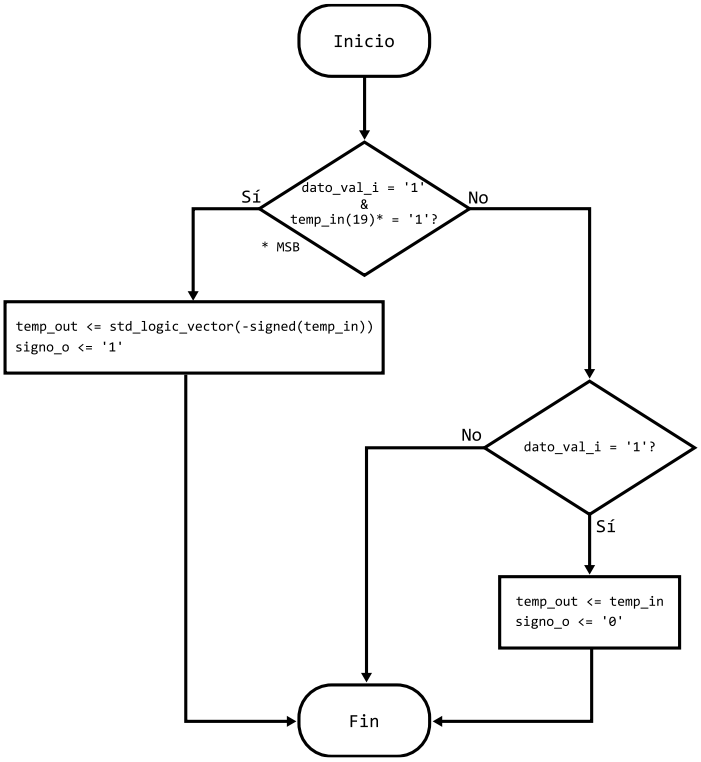


Fig.16. Diagrama de flujo con árboles de decisión para el control del bloque de valor absoluto para temperatura. El diagrama completo se ejecuta en un solo ciclo de reloj.

En el caso en que haya un nuevo dato para procesar (`dato_val_i =’1’`) y el MSB del dato de temperatura de entrada indique que el valor es negativo (`temp_in(19)=’1’`), se

procederá al cálculo del valor absoluto y se indicará que el valor de salida (`temp_out`) es negativo con `signo_o ='1'`.

Por el contrario, si hay un nuevo dato para tratar, pero, no es negativo, se hará un simple *bypass* entre la entrada y la salida de datos de temperatura (`temp_out <= temp_in`) y se indicará que el signo es positivo (`signo_o ='0'`).

Otro detalle para tener en cuenta es el control de la salida `dato_val_o`, que deberá indicar durante un ciclo de reloj al siguiente bloque que ya está disponible el nuevo dato. En este caso, la solución más fácil cosiste en conectar directamente esta salida a la entrada `dato_val_i` de forma incondicional. Así se consigue propagar el pulso al siguiente bloque con un ciclo de reloj de retraso, justo el resultado que se espera. En la Fig.17 se muestra de forma visual este detalle.

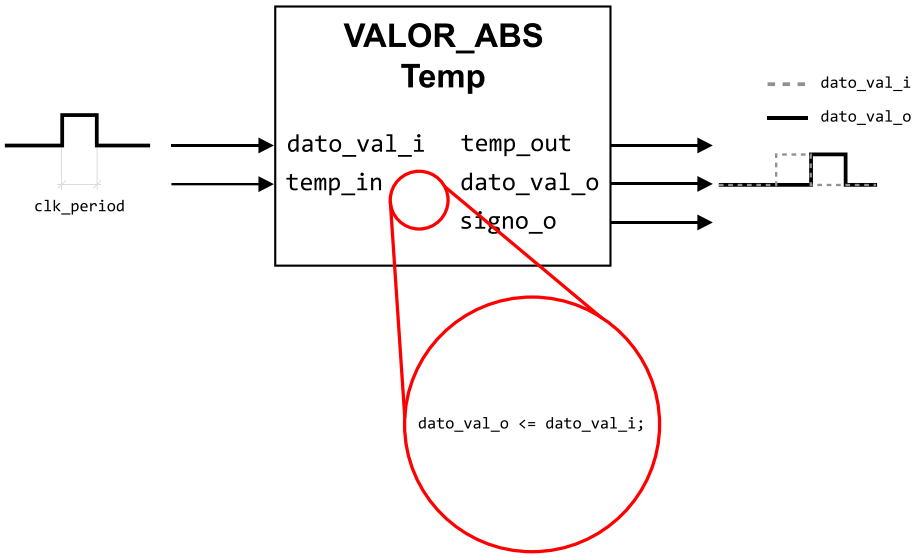


Fig.17. Representación de la propagación de `dato_val_i` un ciclo de reloj retrasado mediante el empleo de `dato_val_o`.

Todo el diseño VHDL de la entidad para el cálculo del valor absoluto está disponible en el archivo `valor_abs.vhd` accesible en: `/src`.

Por último, desde el punto de vista de la Unidad de Control, este bloque pertenece solo al modo temperatura, lo que implica que el bloque previo a este siempre pertenecerá al camino de acondicionamiento de la temperatura y, por tanto, no es necesaria la intervención del *Process P1* para realizar las conexiones entre este bloque y el anterior. En definitiva, las salidas de la entidad para la función de transferencia de la temperatura y las entradas del bloque de valor absoluto están conectadas de forma directa, Fig.18.

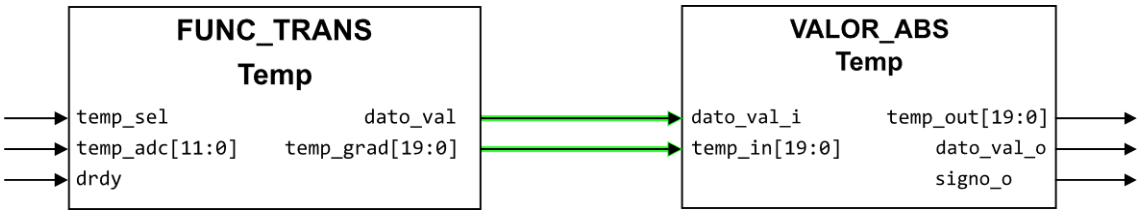


Fig.18. Diagrama de conexiones entre el bloque de función de transferencia para la temperatura y el bloque de valor absoluto.

6. Bloque para la Conversión de Binario Natural a BCD.

Una vez que se tiene la magnitud física adquirida, escalada y acondicionada (valor absoluto, en el caso de la temperatura), será necesario para su representación en los visualizadores de 7 segmentos conocer los dígitos decimales del dato (unidades, decenas, centenas, etc.). Para ello se debe convertir el dato de binario natural al formato conocido como BCD (*Binary-Coded Decimal*). Este formato representa cada dígito decimal con 4 bits, del 0 al 9.

Para realizar la transformación del dato de entrada en binario natural (`binary_in[19:0]`) a BCD se ha diseñado la entidad VHDL `bin2bcd.vhd`. La conversión de binario a BCD requiere de un algoritmo iterativo que requiere de realizar divisiones por 10 e ir almacenando los residuos, pero la implementación de divisiones en hardware es ineficiente.

Para realizar un diseño eficiente, se ha optado por implementar el algoritmo *Double Dabble*. Este algoritmo es muy eficiente en cuanto a recursos, dado que solo realiza operaciones de desplazamiento y suma, por eso también se le conoce como *Shift-and-Add-3*.

6.1. Algoritmo *Double Dabble*.

La idea fundamental de este algoritmo se basa en hacer un desplazamiento lógico a la izquierda (`sll`) sobre dos registros y concatenar el bit más significativo (MSB) del registro binario con el bit menos significativo (LSB) del registro BCD.

Para dimensionar correctamente el registro BDC, es necesario analizar el rango de entrada del registro binario. Dado que `binary` cuenta con 20 bits, el valor máximo representable es 2^{20}- 1 = 1.048.575, lo que requeriría de 7 dígitos decimales para su representación. A pesar de esto, las magnitudes escaladas no alcanzan dicho fondo de escala, por ejemplo, la temperatura solo alcanza el valor 230.535 m°C, siendo este el mayor entre las dos magnitudes. Por esta razón, solo se necesitarán de 6 dígitos decimales, es decir, 24 bits (6 dígitos x 4 bits) para el registro BCD (`bcds_reg[23:0]`).

El proceso se inicia con el primer registro cargado con el valor binario natural (`binary[19:0]`) y el segundo (`bcds_reg[23:0]`) inicializado a cero. En cada iteración, el MSB de `binary` se introducirá en el LSB de este último, tal como se ilustra en la Fig.19.

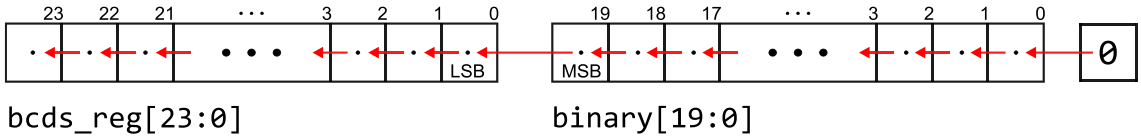


Fig.19. Representación de un desplazamiento lógico a la izquierda concatenado entre el registro binario (20 bits) y el registro BCD (24 bits), con propagación del MSB de `binary` al LSB de `bcds_reg`.

El algoritmo irá haciendo estos desplazamientos de forma iterativa hasta que el contador de iteraciones (`shift_counter`) sea igual al ancho de palabra, en este caso 20. Cuando en alguna de dichas iteraciones, algún grupo de 4 bits del registro `bcds_reg` sea mayor o igual a 5, el algoritmo sumará 3 a dicho grupo. De este modo, al finalizar las 20 iteraciones, en el registro `bcds_reg` estarán los dígitos decimales del dato binario original almacenado inicialmente en `binary`. Los dígitos BCD se devolverán por las salidas `bcd5[3:0]`, `bcd4[3:0]`, `bcd3[3:0]`, `bcd2[3:0]`, `bcd1[3:0]` y `bcd0[3:0]`. El diagrama de flujo del algoritmo se muestra en Fig.20.

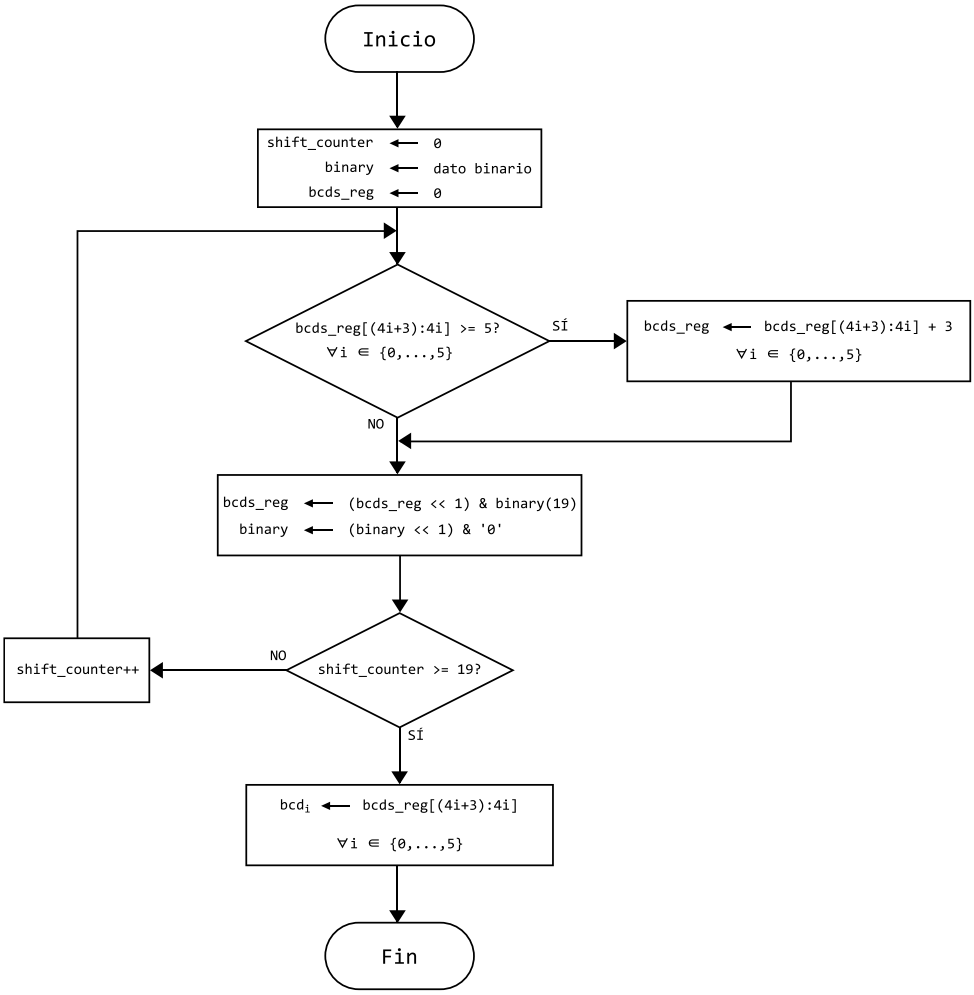


Fig.20. Diagrama de flujo del algoritmo *Double Dabble* (*Shift-and-Add-3*) implementado en la entidad `bin2bcd.vhd` para la conversión de binario natural a BCD.

6.2. Implementación del algoritmo en VHDL.

Para la implementación en VHDL de este algoritmo, se ha optado por diferenciar claramente entre una parte secuencial, para la lógica de control, y otra combinacional, para realizar la corrección (*Add-3*).

Se ha diseñado una Máquina de Estados Finitos (FSM), tanto para realizar la espera activa hasta que hay un nuevo dato disponible para convertir, hasta realizar las operaciones de desplazamiento lógico y concatenación entre registros. Los estados de la FSM son los siguientes:

1. IDLE (Espera): en este estado, el bloque está en reposo monitorizando el valor de la entrada `start`. Cuando `start` valga `'1'`, es una señal proveniente de bloques posteriores, que indica que hay un nuevo dato para convertir. Dicho dato será capturado desde el registro de entrada `binary_in`. Una vez hecho esto, se inicializan los registros de desplazamiento y el contador de iteraciones, y se evoluciona al siguiente estado. Por otra parte, mientras se permanezca en este estado, la salida `ready` se mantiene a `'1'`, indicando que hay una conversión disponible; en cambio, si se evoluciona a `SHIFT`, se pone a `'0'`.

2. SHIFT (Desplazamiento): al comenzar se comprueba el valor del contador, si es igual a 20 (ancho de palabra), significa que se ha finalizado la conversión y se evoluciona al último estado. Si no, se realiza el desplazamiento de ambos registros, introduciendo el MSB del registro binario en el LSB del registro BCD. Además, se incrementa el contador, indicando que se ha realizado una iteración más.

3. DONE (Fin): finalmente, al finalizar se actualiza el registro `bcds_out_reg` con el valor final de la conversión disponible en `bcds_reg`. De este modo, se devuelve la conversión final y se evoluciona al estado inicial para volver a empezar.

Por otra parte, la lógica de corrección del algoritmo (*Add-3*), se ha implementado mediante sentencias concurrentes, es decir, lógica combinacional. Cada ciclo de reloj, se lleva a cabo una monitorización de cada uno de los *nibbles* del registro BCD. Si algún *nibble* es mayor que 4, se aplica una corrección sobre dicho *nibble* sumando 3 al contenido.

Para conectar los valores calculados de forma combinacional (corrección *Add-3*) del registro BCD con los valores obtenidos de forma secuencial (desplazamiento concatenado) en la FSM, se emplea una cadena de dependencias combinacionales entre registros y señales, tal y como se ilustra en la Fig.21.

En el primer flanco se actualiza el valor del registro `bcds` con el valor `bcds_next`, calculado en el ciclo anterior. Después, al valor de `bcds` se le aplica de forma combinacional la corrección *Add-3*, cuyo resultado se propaga por la señal `bcds_reg`. Finalmente, si la FSM se encuentra en el estado `SHIFT`, se ejecuta el desplazamiento lógico a la izquierda sobre la señal `bcds_reg`, estableciendo el valor futuro de la señal `bcds_next` antes de la llegada del siguiente flanco.

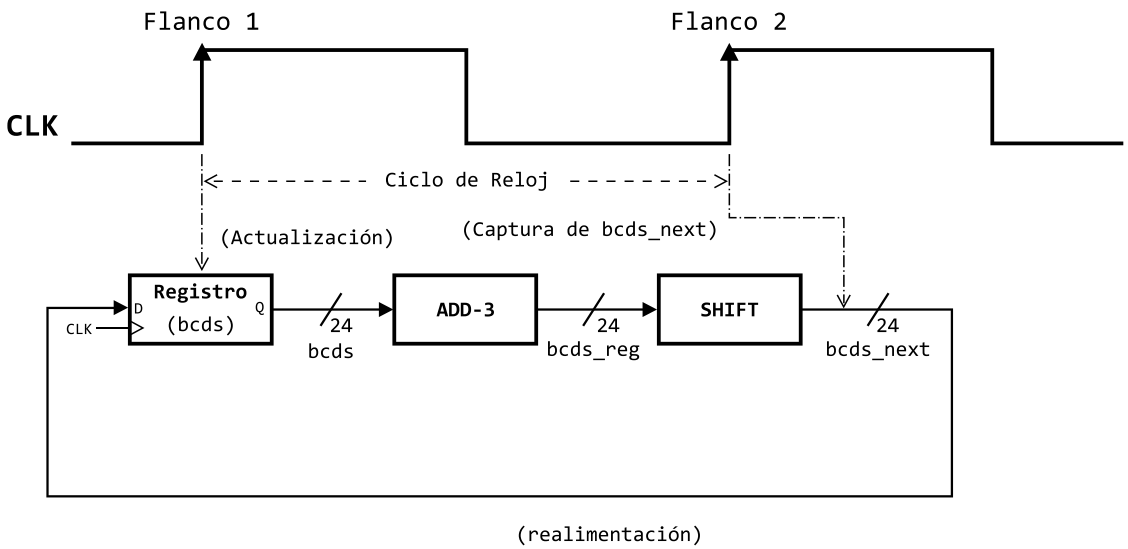


Fig.21. Diagrama temporal conceptual del flujo de datos para el cálculo del registro BCD, mostrando la actualización síncrona del registro `bcds` y la propagación combinacional de las señales intermedias a través de las etapas de corrección (`ADD-3`) y desplazamiento (`SHIFT`) dentro de un ciclo de reloj.

Por otra parte, una vez que el contador de iteraciones haya llegado al límite ($N=20$), la FSM evolucionará al estado `DONE`. Solo en este punto se actualizarán los valores de salida con el valor calculado de la conversión para evitar que se propaguen los cambios intermedios del algoritmo a los puertos de salida.

Para realizar esto, el diseño emplea un registro dedicado (`bcds_reg_out`) para separar la lógica interna de desplazamiento y corrección con los valores de salida. Cuando se esté en el último estado, el valor de `bcds_next` obtenido en el último ciclo, será capturado en este registro de salida.

A partir de este registro estable, en cada ciclo de reloj, los puertos de salida `bcd0` a `bcd5` serán actualizados de forma combinacional mediante la asignación de cada uno de los *nibbles* de `bcds_reg_out` a su correspondiente salida, tal y como se muestra a continuación:

bin2bcd.vhd

```
-- Salidas
 bcd5 <= bcds_out_reg(23 downto 20);
 bcd4 <= bcds_out_reg(19 downto 16);
 bcd3 <= bcds_out_reg(15 downto 12);
 bcd2 <= bcds_out_reg(11 downto 8);
 bcd1 <= bcds_out_reg(7 downto 4);
 bcd0 <= bcds_out_reg(3 downto 0);
```

Para sintetizar y simplificar el diseño de toda la descripción hardware, la interfaz de entrada/salida se ilustra en la Fig.22. Para más profundidad, todo el código fuente VHDL de la entidad `bin2bcd.vhd` se encuentra disponible en la siguiente carpeta: `/src`.

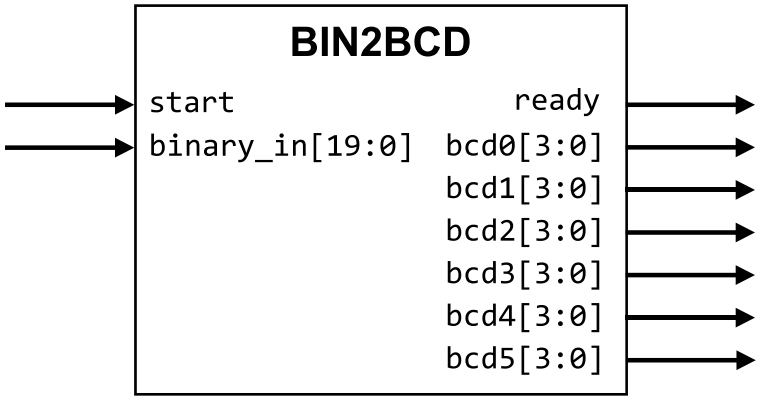


Fig.22. Interfaz de entrada/salida del bloque para la conversión de binario natural a BCD. Se han omitido las señales de reloj y *reset* por claridad.

6.3. Integración con la Unidad de Control y Multiplexado de Entradas.

Finalmente, desde la perspectiva de la integración del bloque en la entidad superior (`adc_top_level.vhd`), el bloque `bin2bcd.vhd` se ha instanciado como un recurso compartido. Es decir, en lugar de duplicar los recursos hardware para la conversión de las etapas de temperatura y voltaje, se emplea un único convertidor para ambas magnitudes. Por este motivo, será la Unidad de Control (*Process P1*) la encargada de multiplexar las señales de entrada según el modo seleccionado.

Cuando el modo temperatura está seleccionado (`temp_sel = '1'`), la unidad de control debe enrutar la salida de datos del bloque de valor absoluto de la temperatura (`temp_out`) a la entrada de datos `binary_in`. Para ello, se emplea la señal intermedia `temp_abs` (conectada directamente a `temp_out`) que se conectará con la señal `binary_in_bcd` (conectada directamente con `binary_in`). Asimismo, la señal de activación `dato_val_o` deberá conectarse a la entrada `start`, en este caso, empleando las señales intermedias `dato_val_temp_abs` y `start_bcd`. La Fig.23 ilustra de forma gráfica la conexión entre lo bloques multiplexada por la unidad de control.

En cambio, cuando se activa el modo voltaje (`temp_sel = '0'` y `volt_sel = '1'`), las salidas del bloque de función de transferencia para el voltaje deben ser redirigidas a las entradas del bloque para la conversión a BCD. La salida de datos `volt_mv` se enruta a la entrada `binary_in`, para ello se emplean las señales intermedias `volt_mv` y `binary_in_bcd`. Por último, la salida `dato_val` se conectará con la entrada `start` por medio de las señales `dato_val_volt` y `start_bcd`, Fig.24.

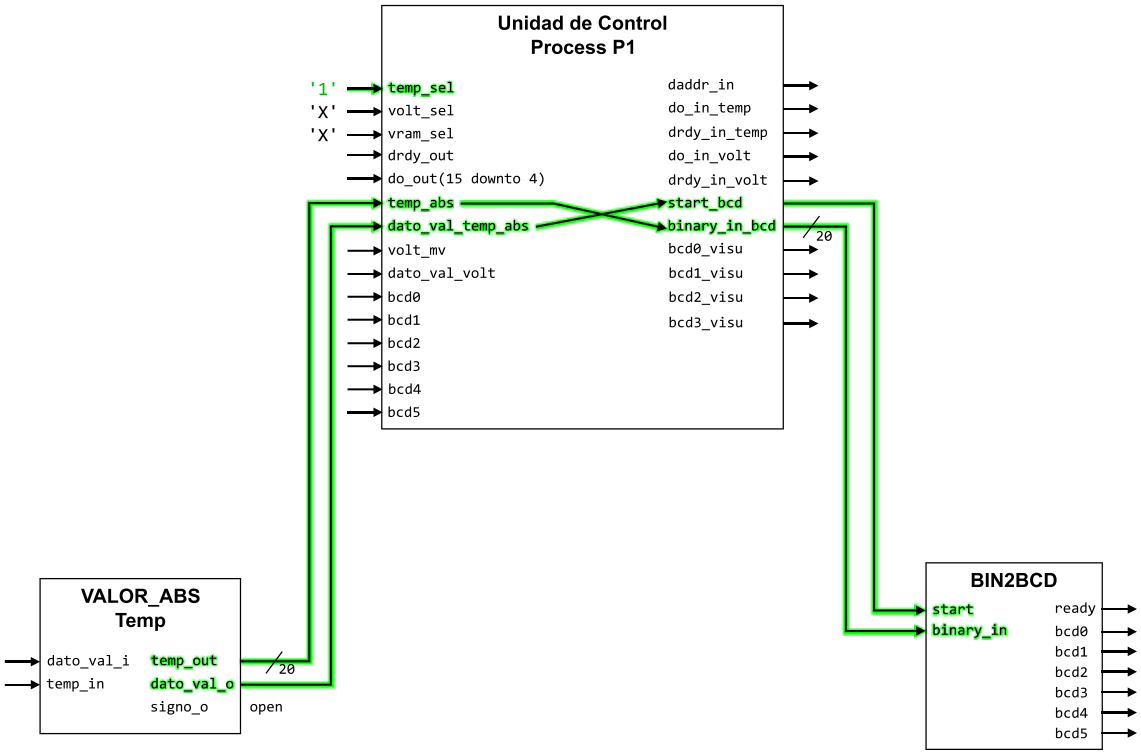


Fig.23. Diagrama de bloques con conexiones multiplexadas mediante la Unidad de Control (*Process P1*) para el modo temperatura.

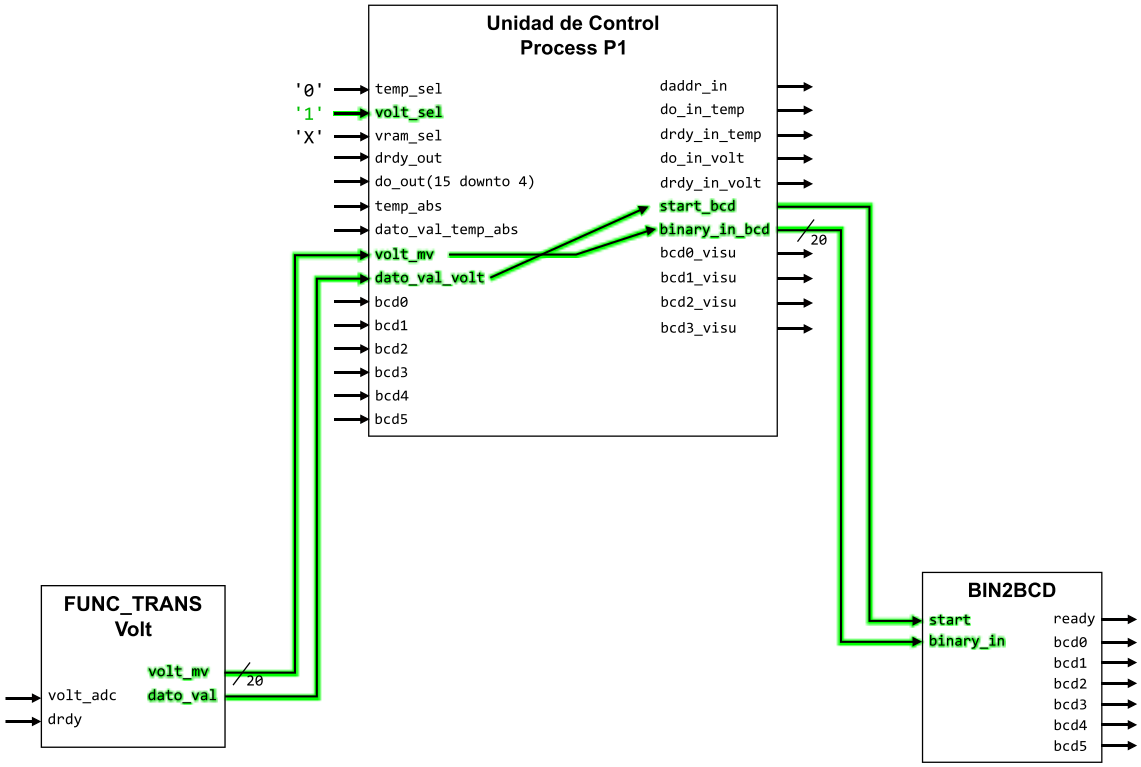


Fig.24. Diagrama de bloques con conexiones multiplexadas mediante la Unidad de Control (*Process P1*) para el modo voltaje.

7. Bloque para el Control de la Visualización.

El último bloque en esta cadena de procesado es la entidad `visu7seg.vhd`. Su funcionalidad consiste en capturar los dígitos BCD y representarlos en el visualizador 7 segmentos de la FPGA, Fig.25.

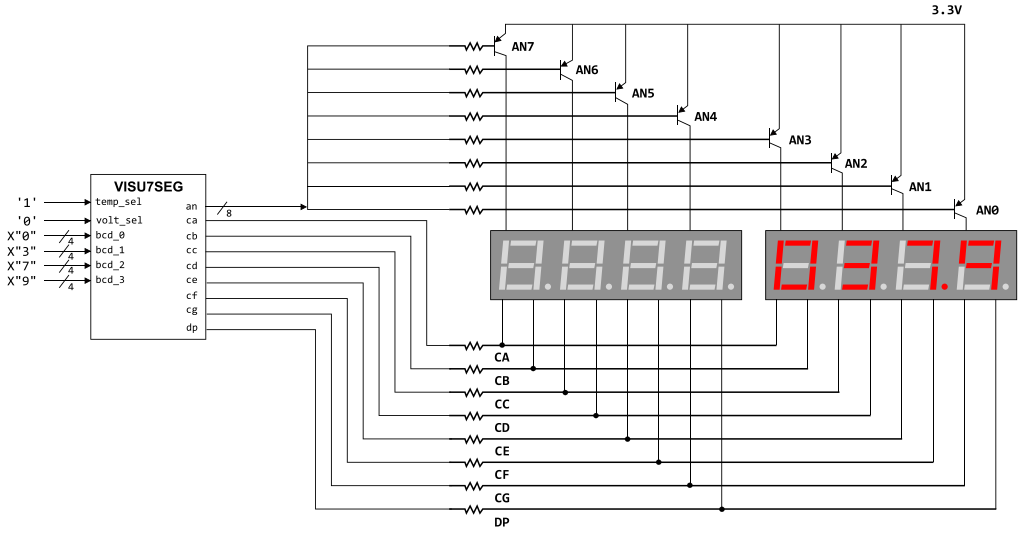


Fig.25. Esquema de interconexión entre la entidad `visu7seg.vhd` y el visualizador de la placa. Se ilustra un ejemplo funcional donde el bloque decodifica y representa una lectura de temperatura (37.9°C), mostrando la relación entre los puertos digitales de salida y los segmentos del hardware.

Antes de realizar el diseño del bloque, es necesario entender el funcionamiento del visualizador 7 segmentos. La placa Nexys4 DDR dispone de consta de 8 dígitos con una estructura de Ánodo Común, donde los terminales de control de los segmentos (cátodos) comparten un mismo bus eléctrico para todos los dígitos (Fig.26). Debido a este recurso compartido, no es posible encender todos los dígitos simultáneamente con valores diferentes, deben activarse los ánodos de forma secuencial (uno a uno) mientras se sitúa el dato deseado en las líneas de los cátodos.

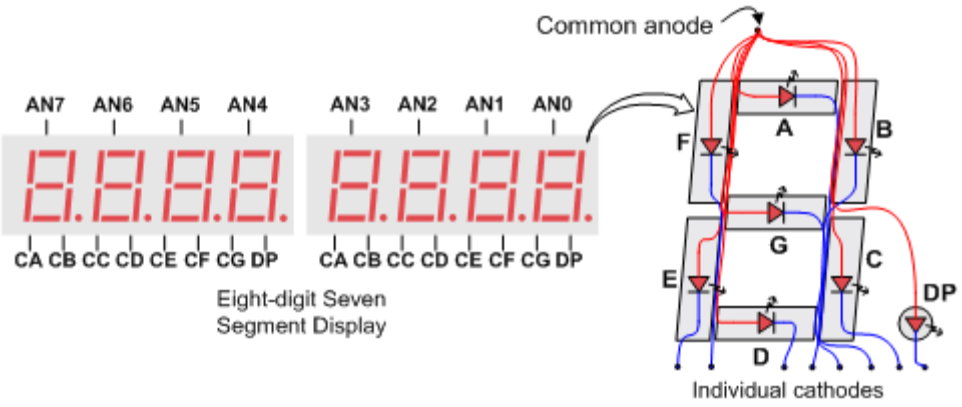


Fig.26. Circuito del visualizador 7 segmentos de 8 dígitos en ánodo común, extraído de [27].

En la Fig.27 se ilustra la representación de solo uno de los dígitos BCD. Pude observarse que tanto ánodos como cátodos tienen lógica negativa. Para representar el dígito BCD 3 en el tercer dígito del visualizador, debe forzarse `'0'` el bit 2 de la salida `an[7:0]` (AN2), y situar el valor 3 en formato 7 segmentos (`"00001101"`) en los puertos correspondientes a los cátodos, de `ca` a `dp`.

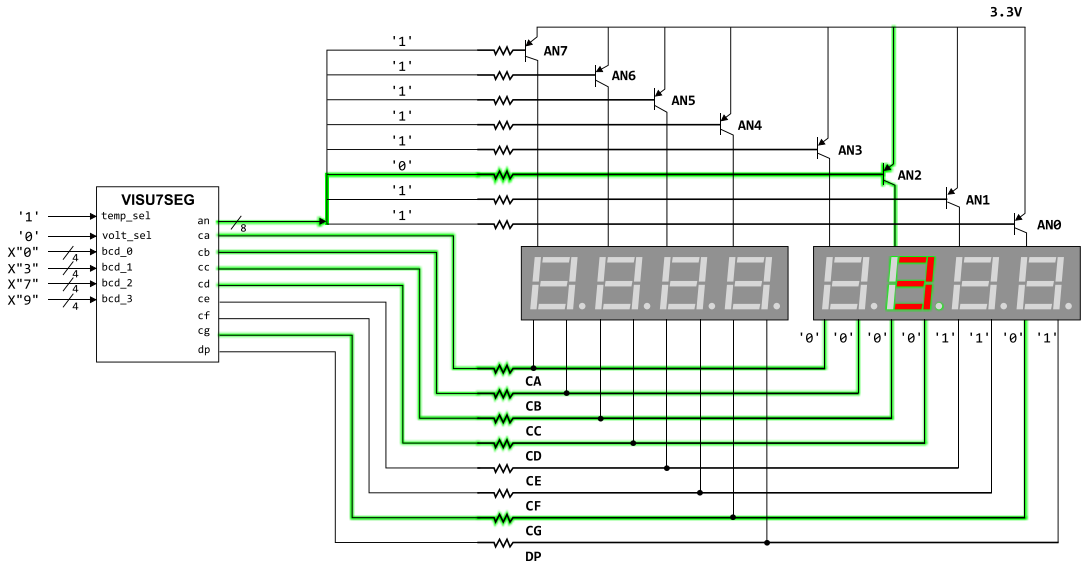


Fig.27. Diagrama conceptual de la representación de un único dígito BCD en el visualizador 7 segmentos, las señales activas (lógica negativa) aparecen resaltadas en verde.

Por otra parte, para solucionar la cuestión del uso compartido de los cátodos, el propio fabricante indica la posibilidad de realizar una secuencia temporal en las señales de los ánodos según el patrón de la Fig.28. El tiempo de activación de cada una de las señales debe situarse entre 0.25 y 4 ms.

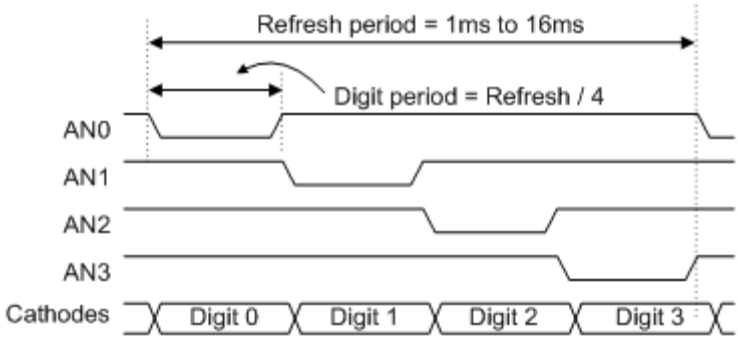
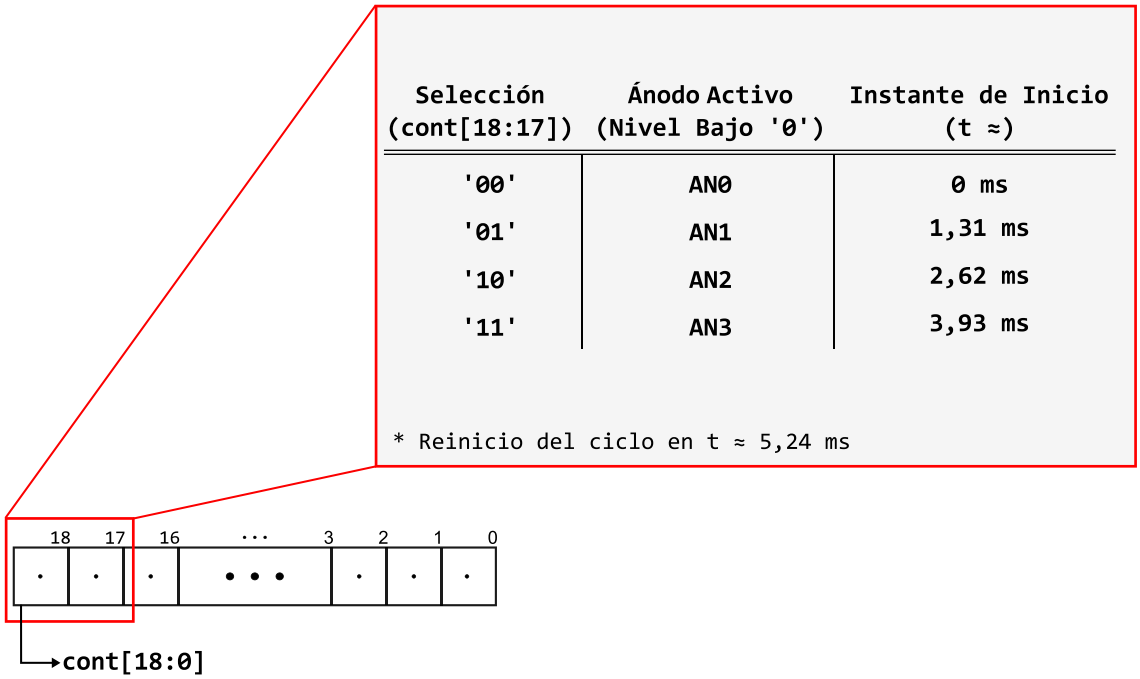


Fig.28. Diagrama temporal del control sobre los ánodos para la visualización simultánea de 4 dígitos, extraído de [27].

7.1. Implementación del Multiplexado Temporal de los Ánodos.

Para abordar el problema de la secuencia temporal en los ánodos, se ha decidido emplear un contador de 19 bits (`cont`, Fig.29.), de modo que las distintas combinaciones de los bits 18 y 17 controlen el ánodo activo.

Al emplear el bit 17 como LSB se consigue cambiar de ánodo cada 2^{17} ciclos de reloj, lo que supone un tiempo aproximado de 1.31 ms de refresco entre cada dígito, tiempo suficiente para que el ojo humano no detecte parpadeos entre cambios.



Selección (cont[18:17])	Ánodo Activo (Nivel Bajo '0')	Instante de Inicio (t ≈)
'00'	AN0	0 ms
'01'	AN1	1,31 ms
'10'	AN2	2,62 ms
'11'	AN3	3,93 ms

* Reinicio del ciclo en t ≈ 5,24 ms

Fig.29. Relación temporal entre los bits de selección (`cont[18:17]`) y la activación secuencial de los ánodos. La tabla detalla los intervalos de tiempo asignados a cada dígito, resultando en un ciclo de refresco completo de 5,24 ms.

Para implementar esta lógica en VHDL, se emplea la señal `an[7:0]` cuyos bits corresponden a cada uno de los ánodos (AN7 a AN0). Dependiendo de los valores de `cont[18:17]`, se activará uno u otro ánodo. Los ánodos correspondientes a los 4 dígitos superiores (AN7 a AN4), siempre se mantendrán apagados (`'1'`).

visu7seg.vhd

```
an(3 downto 0) <= "1110" when (cont(18 downto 17)= "00" and reset = '0') else
                  "1101" when (cont(18 downto 17)= "01" and reset = '0') else
                  "1011" when (cont(18 downto 17)= "10" and reset = '0') else
                  "0111" when (cont(18 downto 17)= "11" and reset = '0') else
                  "1111";
an(7 downto 4) <= "1111"; -- No mostramos segmento superior nunca
```

7.2. Estabilización de la Lectura (*Sample* & *Hold*).

Uno de los principales problemas enfrentados durante el diseño del bloque fue el fenómeno de los segmentos fantasma. Debida a la alta frecuencia de muestreo del XADC (961,5 kHz), los dígitos BCD a la entrada del bloque `visu7seg.vhd` presentan un alto grado de variabilidad.

Aunque siempre hay un dato que permanece un tiempo superior al resto, las capacidades parásitas en los diodos LED provocan que segmentos pertenecientes a otros datos con menor tiempo de persistencia, se iluminen con una intensidad inferior (segmentos fantasma). Este efecto es lo bastante notable para impedir la correcta lectura de los datos en el visualizador (Fig.30).

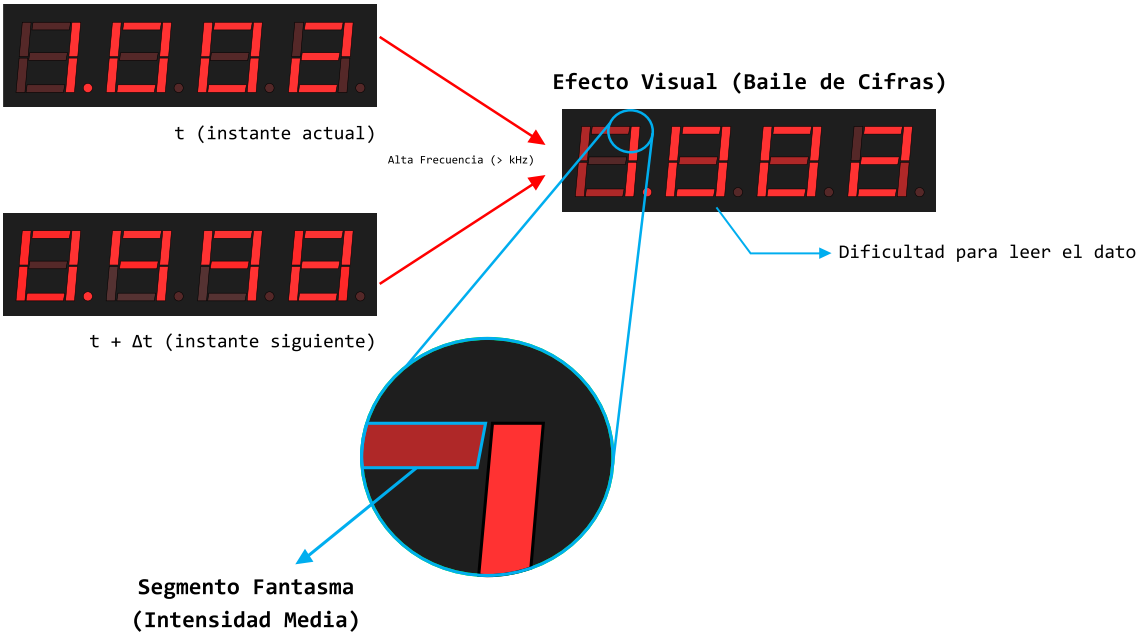


Fig.30. Representación visual del problema de los segmentos fantasma provocados por la conmutación de elevada frecuencia entre varios valores (baile de cifras). En este caso, a modo de ejemplo, se representa la alternancia entre dos valores cercanos de voltaje, 0.998 V y 1.002 V.

Para solucionar este problema, se ha decidido emplear variables internas para los dígitos BCD (`bcd_int0` a `bcd_in3`), que amortigüen la velocidad de actualización de los dígitos BCD de entrada. Esto se consigue actualizando las señales internas (`bcd_int_i`) con los puertos de entrada (`bcd_i`), a una frecuencia muy inferior a la tasa de muestreo del XADC.

La frecuencia de actualización de los dígitos BCD se ha decidido que sea de unos 3 Hz, más que suficiente para evitar el efecto de los segmentos fantasma. Esto obliga a emplear un contador de 25 bits (10).

$$f = \frac{f_{clk}}{2^{25}} = \frac{100\ \text{MHz}}{2^{25}} \approx 2{,}98\ \text{Hz} \tag{10}$$

Para ahorrar recursos, se opta por extender el contador `cont[18:0]` a 25 bits (Fig.31) para emplear un solo registro para dos tareas distintas, la multiplexación temporal de los ánodos del visualizador y la estabilización en la lectura de los dígitos BCD.

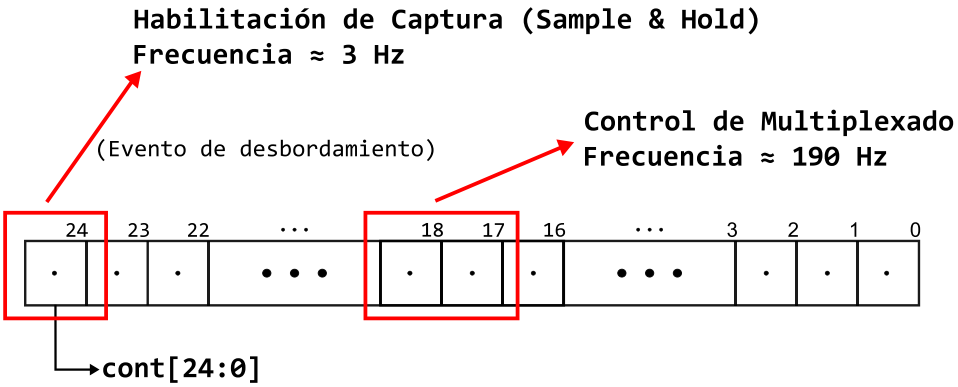


Fig.31. Arquitectura del contador multifuncional cont. Los bits intermedios (17-18) gestionan el barrido rápido del visualizador a 190 Hz, mientras que el desbordamiento del MSB (bit 24) inicia la captura estable de datos a 3 Hz.

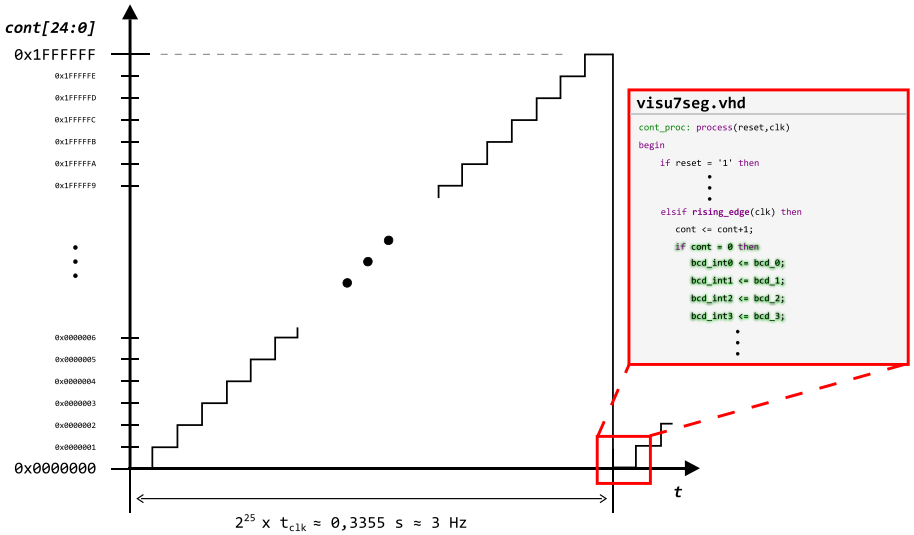


Fig.32. Cronograma de la lógica de estabilización visual. Se ilustra cómo el desbordamiento del contador de 25 bits genera el evento de habilitación (3 Hz) que dispara la captura de los datos de entrada (*Sample & Hold*) en el código VHDL, evitando la fluctuación de cifras en el visualizador.

Para capturar los datos no se puede emplear directamente el MSB del contador, dado que este bit permanece activo durante un tiempo igual a 2^{24} ciclos. Si se hiciera de esta forma, se estarían actualizando los dígitos BCD durante ese tiempo, provocando el mismo problema que al inicio. En cambio, se empleará el evento de desbordamiento del contador, es decir, se detectará el momento en el cual el contador vale cero. Este evento solo ocurre

durante un ciclo de reloj, luego es perfecto para conseguir una captura estable de los dígitos.

En la Fig.32 se ilustra el evento de desbordamiento del contador y, además, se relaciona dicho evento con el código VHDL necesario para capturarlo y la actualización correspondiente de los dígitos BCD internos con los externos en dicho ciclo de reloj.

7.3. Conversión de BCD a 7 Segmentos (Cátodos).

Una vez se tienen los dígitos BCD estables, se debe realizar la conversión a 7 segmentos para activar los cátodos correspondientes para su visualización en el 7 segmentos. Para ello, se empleará la señal `s_int` que almacenará el dígito BCD que toque representar en ese momento, reutilizando los bits 17 y 18 de `cont` para dicho propósito:

visu7seg.vhd

```
s_int <= bcd_int0 when (cont(18 downto 17)= "00") else
         bcd_int1 when (cont(18 downto 17)= "01") else
         bcd_int2 when (cont(18 downto 17)= "10") else
         bcd_int3;
```

Una vez se tenga el dígito BCD en la señal `s_int`, se usará para realizar una asignación selectiva de señal (`with … select`). Es decir, dependiendo de su valor, la señal `visu` contendrá el código 7 segmentos del dígito. De esta forma, se consigue implementar un decodificador BCD a 7 segmentos mediante una tabla de búsqueda:

visu7seg.vhd

```
  with s_int select
  visu <= "0000001" when "0000",  -- 0
          "1001111" when "0001",  -- 1
          "0010010" when "0010",  -- 2
          "0000110" when "0011",  -- 3
          "1001100" when "0100",  -- 4
          "0100100" when "0101",  -- 5
          "0100000" when "0110",  -- 6
          "0001111" when "0111",  -- 7
          "0000000" when "1000",  -- 8
          "0000100" when "1001",  -- 9
          "1111111" when others;  -- apagado
```

Con la decodificación a 7 segmentos realizada, los bits de la señal visu se asignan de forma concurrente a los puertos de salida correspondientes a los cátodos, salvo el cátodo del punto (`dp`) que debe operarse de forma especial.

7.4. Gestión Dinámica del Punto Fijo Decimal.

Debido a la discrepancia entre los formatos de representación numérica de la temperatura (`XXX.X`) y el voltaje (`X.XXX`), el cátodo correspondiente al punto fijo del visualizador debe activarse de forma condicional (`dp <= ‘0’ when ... else ...`):

visu7seg.vhd

```
dp <= '0' when (temp_sel = '1' and cont(18 downto 17)= "01" and reset = '0') else
      '0' when (volt_sel = '1' and temp_sel = '0' and cont(18 downto 17)= "11"
          and reset = '0') else
      '1';
```

Cuando el modo temperatura esté seleccionado (`temp_sel = ‘1’`), el punto fijo deberá activarse en el segundo dígito del visualizador (`cont(18 downto 17) = “01”`), Fig.33 (a). En cambio, en el modo voltaje (`temp_sel = ‘0’` y `volt_sel = ‘1’`), el punto fijo se activará en el último dígito (`cont(18 downto 17) = “11”`), Fig.33 (b).

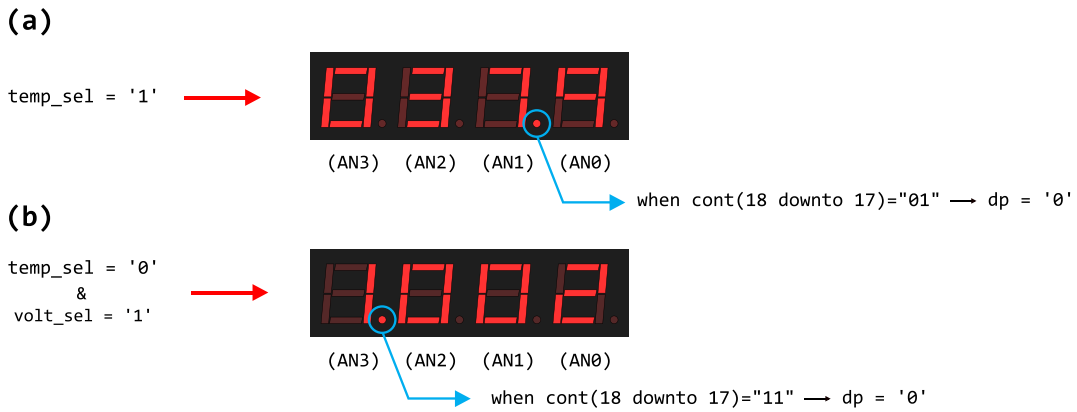


Fig.33. (a) Gestión del encendido del punto fijo del visualizador para el modo temperatura. (b) Gestión del encendido del punto fijo para el modo voltaje.

7.5. Gestión del Estado de Reposo.

Otro punto para tener en cuenta a la hora del diseño es la gestión del estado de reposo, cuando los modos temperatura y voltaje están desactivados (`temp_sel = ‘0’` y `volt_sel = ‘0’`). Aquí se presentan varias posibles soluciones para atajar el problema.

En un primer momento se podría pensar en representar todos los dígitos con un cero (`0000`), el problema de esta opción es su facilidad para ser interpretada como falsas lecturas de ceros, tanto de temperatura como de voltaje (Fig.34 (a)).

Otra posible solución podría ser no representar ningún dígito, Fig.34 (b), aunque su principal inconveniente es que el usuario podría pensar que el sistema está apagado, si no está lo suficiente familiarizado con el mismo.

Por último, la opción implementada ha consistido en representar los segmentos G de cada dígito, mostrando una serie de guiones (`----`). De esta forma se consigue evitar confusiones con falsas lecturas o malentendidos con el estado de encendido del sistema, Fig.34 (c).

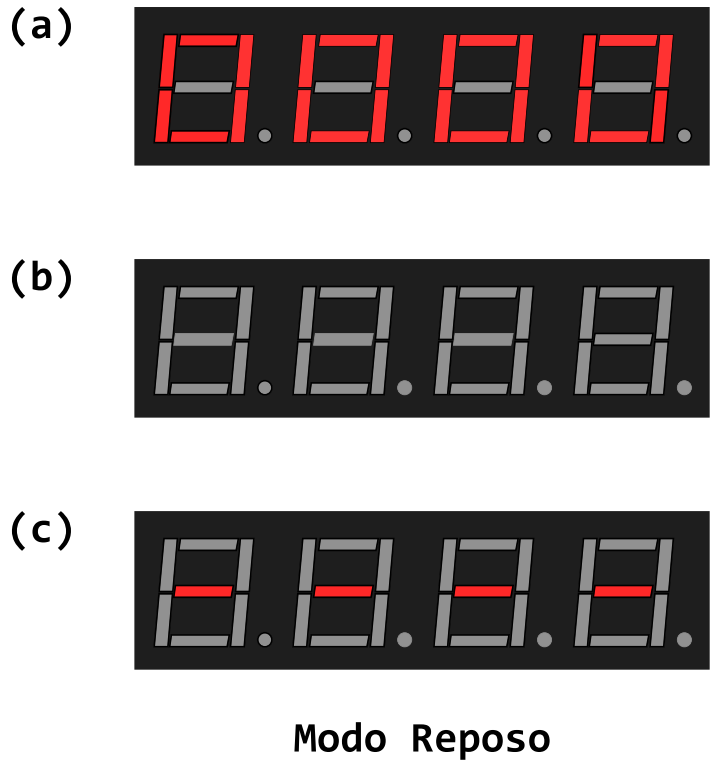


Fig.34. Ilustración de posibles soluciones a la gestión del modo reposo del sistema de visualización de Temperatura y Voltaje: (a) dígitos con ceros, (b) visualizador apagado y (c) guiones en cada dígito.

Para implementar esta solución se ha empleado el valor `0xA` en las señales internas `bcd_int`, de modo que cuando haya un evento de *reset* o se esté en modo reposo, se asigne esté valor a dichas señales. Además, se ha modificado la asignación selectiva de la señal `visu` para que, cuando `s_int` sea `0xA`, se active exclusivamente el cátodo central (*cg*), generando el patrón visual del guión:

visu7seg.vhd

```
cont_proc: process(reset,clk)
begin
  if reset = '1' then
    -- ...
    bcd_int0 <= X"A"; -- Valor por defecto al Reset
    -- ...
    bcd_int3 <= X"A";
  elsif rising_edge(clk) then
    -- ...
    if temp_sel = '0' and volt_sel = '0' then
```

```
        bcd_int0 <= X"A"; -- Forzar guiones en modo Reposo
        -- ...
        bcd_int3 <= X"A";
    end if;
    -- ...
  end if;
end process;

-- Decodificador de salida
with s_int select
  visu <= "0000001" when "0000", -- 0
          -- ...
          "1111110" when "1010", -- A: Gestión del modo reposo -> [----]
          "1111111" when others; -- Apagado
```

7.6. Integración con la Unidad de Control.

Para integrar los datos calculados en la interfaz de usuario (visualizador), se requiere resolver una discrepancia dimensional entre la capacidad de procesamiento interno y la capacidad de visualización. Mientras que el bloque `bin2bcd.vhd` genera un vector de salida de 6 dígitos para cubrir el rango completo de las variables (hasta centenares de miles de unidades en m°C, el visualizador físico de la FPGA está restringido a 4 caracteres. Para conciliar esta diferencia, la Unidad de Control implementa una lógica de enrutamiento selectivo que actúa como una ventana deslizante sobre el bus de datos, eligiendo qué grupo de cuatro dígitos se transmite al bloque `visu7seg.vhd` en función del modo seleccionado.

En el modo de medición de temperatura, se debe ofrecer una lectura con resolución de décimas de grado (0,1 °C). Dado que el dato interno está expresado en miligrados, el sistema descarta los dos dígitos menos significativos (unidades y decenas de miligrado, BCD_1 y BCD_0) y enruta los dígitos intermedios (BCD_5 a BCD_2) hacia el visualizador. Esto permite que un valor interno de alta precisión, como 37950 m°C (`0x0943E`), se visualice por el usuario como `037.9` (Fig.35).

Por el contrario, en el modo voltaje, donde el rango es menor (0 a 3000 mV), se requiere visualizar la integridad del valor entero. En este caso, el enrutamiento se alinea con los bits menos significativos, conectando directamente los dígitos BCD_3 a BCD_0 al visualizador, descartando los bits superiores BCD_5 y BCD_4. En la Fig.36 se muestra el ejemplo para una entrada de 1002 mV (`0x003EA`) que se visualiza como `1.002`, aprovechando la resolución completa del conversor.

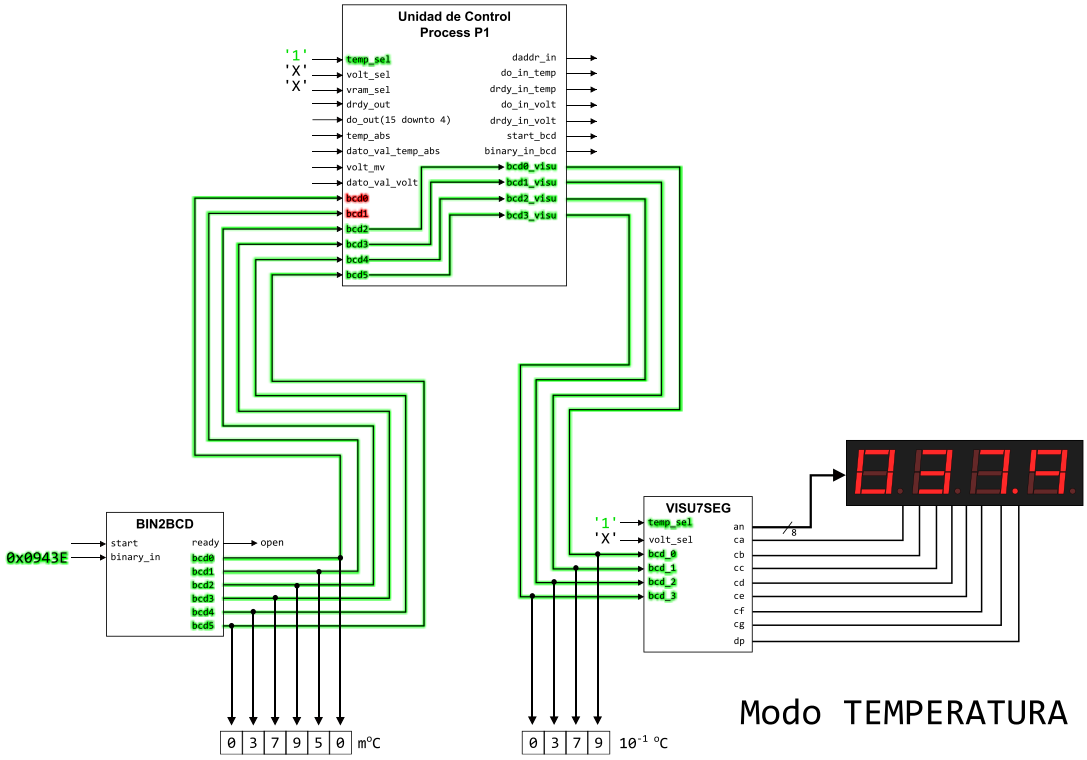


Fig.35. Esquema del enrutamiento dinámico de datos gestionado por la Unidad de Control para el modo Temperatura, se seleccionan los dígitos superiores (BCD_5 a BCD_2) para ajustar la escala a décimas de grado.

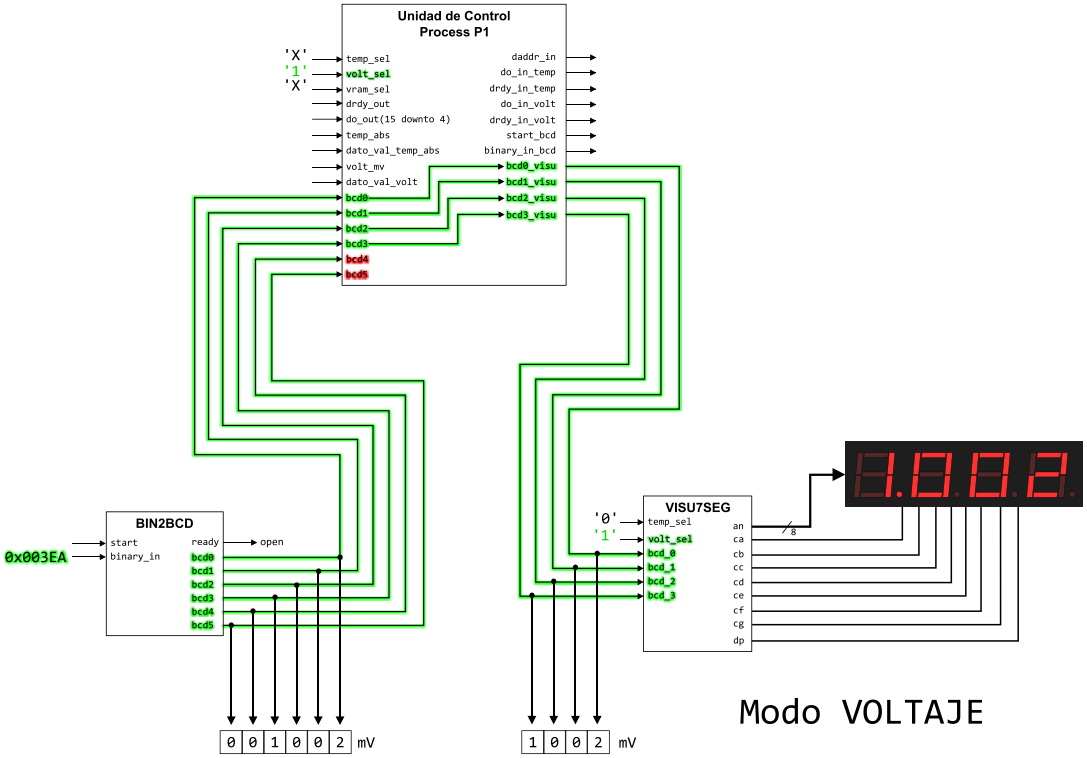


Fig.36. Esquema del enrutamiento dinámico de datos gestionado por la Unidad de Control para el modo Voltaje, se seleccionan los dígitos inferiores (BCD_3 a BCD_0) para visualizar el valor absoluto en milivoltios.

Todos los códigos fuente VHDL de la entidad `visu7seg.vhd` está disponible en la carpeta: `/src`.

8. Unidad de Control (*Process P1*).

Esta unidad es realmente un proceso secuencial que se encarga de que los datos entre recursos compartidos lleguen en forma y momento adecuados. Para ello la unidad realiza un enrutado dinámico de las entradas y salidas de los distintos bloques dependiendo del modo seleccionado, tal y como se ha ido mencionando en los subapartados anteriores.

Aparte de esto, esta unidad también tiene como funcionalidad asignar la dirección del registro de estado del XADC que se desea leer, asignando estas direcciones a `daddr_in` según los valores indicado en la Tabla 2.

La implementación en VHDL se ha realizado mediante una estructura condicional `if-elsif` dentro del proceso secuencial manteniendo las prioridades asignadas en la Tabla 1. La Fig.8 ilustra a la perfección la lógica implementada, con la diferencia de la necesidad de añadir el enrutado de los puertos de los bloques mediante señales intermedias.

El código VHDL correspondiente al *Process P1* puede consultarse en el archivo fuente `adc_top_level.vhd` disponible en: `/src`.

9. Integración en la Entidad Superior (*adc_top_level.vhd*).

Finalmente, la entidad `adc_top_level.vhd` se encarga instanciar todas las entidades anteriores, definir las señales intermedias que el *Process P1* empleará para realizar el enrutado dinámico, albergar el propio proceso secuencial de la Unidad de Control, y servir como interfaz de entrada/salida del sistema. En la Fig.37 se define la interfaz de este bloque, para más detalle se puede consultar la Fig.3.

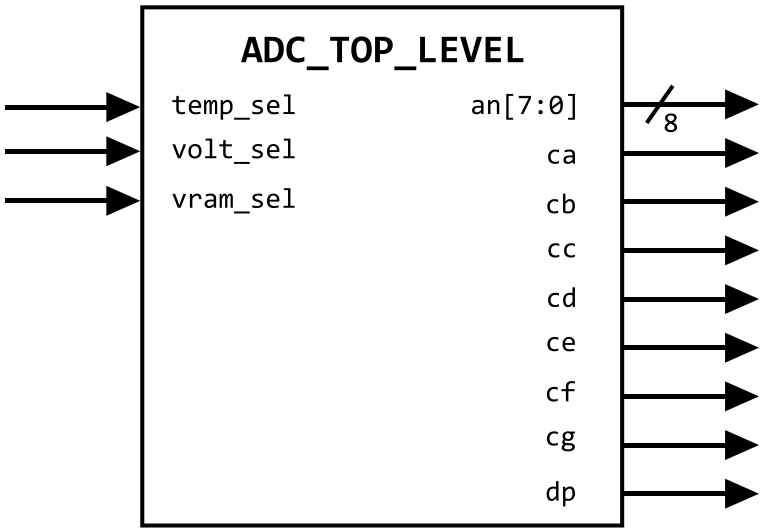


Fig.37. Interfaz de entrada/salida de la entidad superior `adc_top_level.vhd`. Se han omitido las señales de reloj y *reset* por simplicidad.

10. Interfaz Física y Visualización en la FPGA.

Finalmente, en este subapartado se ilustra con imágenes reales como se ha mapeado el control del sistema físicamente en la FPGA. Además, se muestran los modos de reposo, temperatura y voltaje en ejecución real en la FPGA.

10.1. Mapeo de Entradas/Salidas.

La Fig.38 muestra el despliegue físico del sistema sobre la FPGA, indicando la situación de los interruptores asociados a las señales de control y el visualizador 7 segmentos. Se ha asignado el interruptor SW0 al *reset* del procesador, SW1 a la señal `temp_sel`, SW2 a `volt_sel` y SW3 a `vram_sel`. Los resultados se muestran en los 4 dígitos menos significativos del visualizador de 7 segmentos.

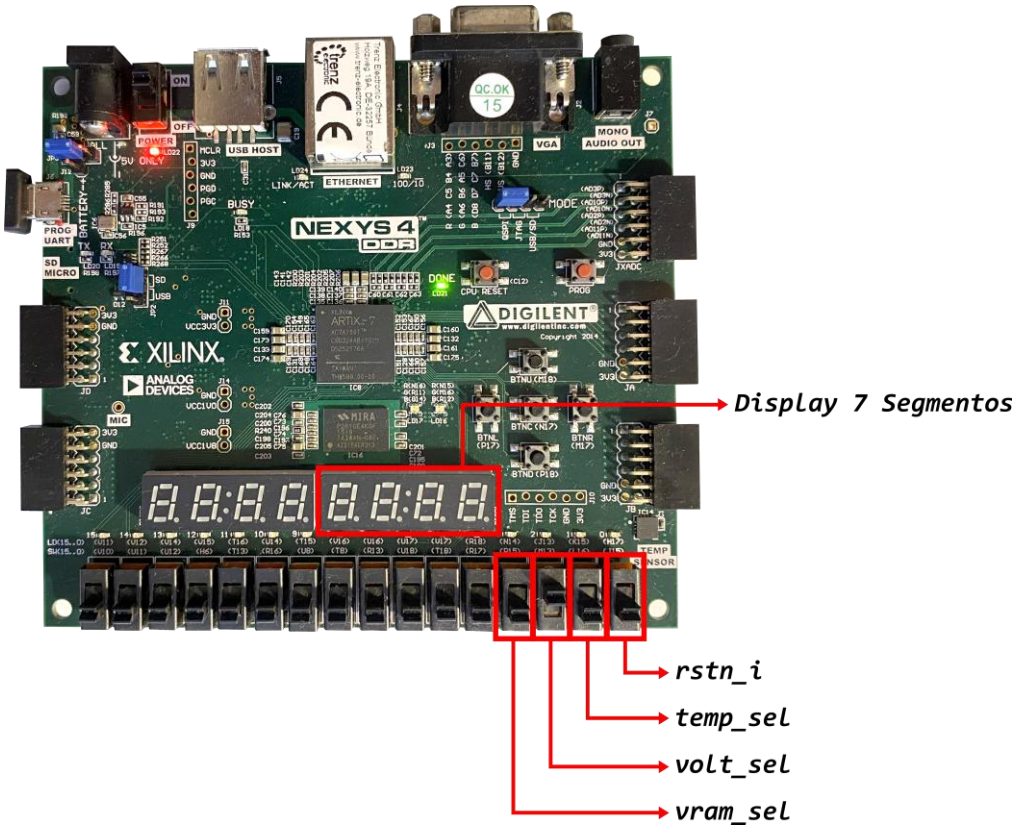


Fig.38. Despliegue de la interfaz de entrada/salida del sistema de visualización multiplexada de señales en la placa Nexys4 DDR.

10.2. Modos de Funcionamiento.

Dependiendo de la configuración de los interruptores de control, el sistema tendrá un modo de funcionamiento u otro:

1. Modo Reposo: este modo se ejecuta cuando las señales `temp_sel` y `volt_sel` valgan `'0'`. La Fig.39 muestra una imagen real de este modo de funcionamiento.

2. Modo Temperatura: al activar `temp_sel`, el ADC muestrea el canal del sensor interno de temperatura. En la Fig.40 se muestra una imagen real del modo temperatura.

3. Modo Voltaje: al activar `volt_sel`, si `temp_sel` está desactivado, se entrará en el modo voltaje. Dependiendo del valor de `vram_sel`, el ADC muestreará el voltaje de alimentación o el de la VRAM. En la Fig.41 se muestra una imagen real del modo voltaje.

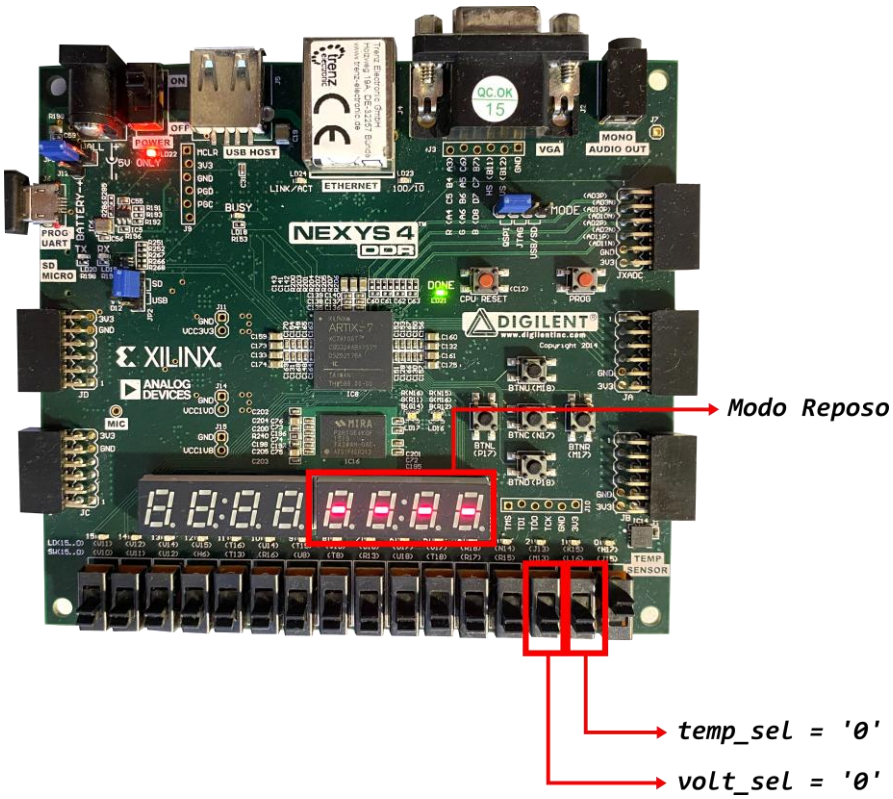


Fig.39. Imagen real del Modo Reposo del sistema de visualización multiplexada de las señales.

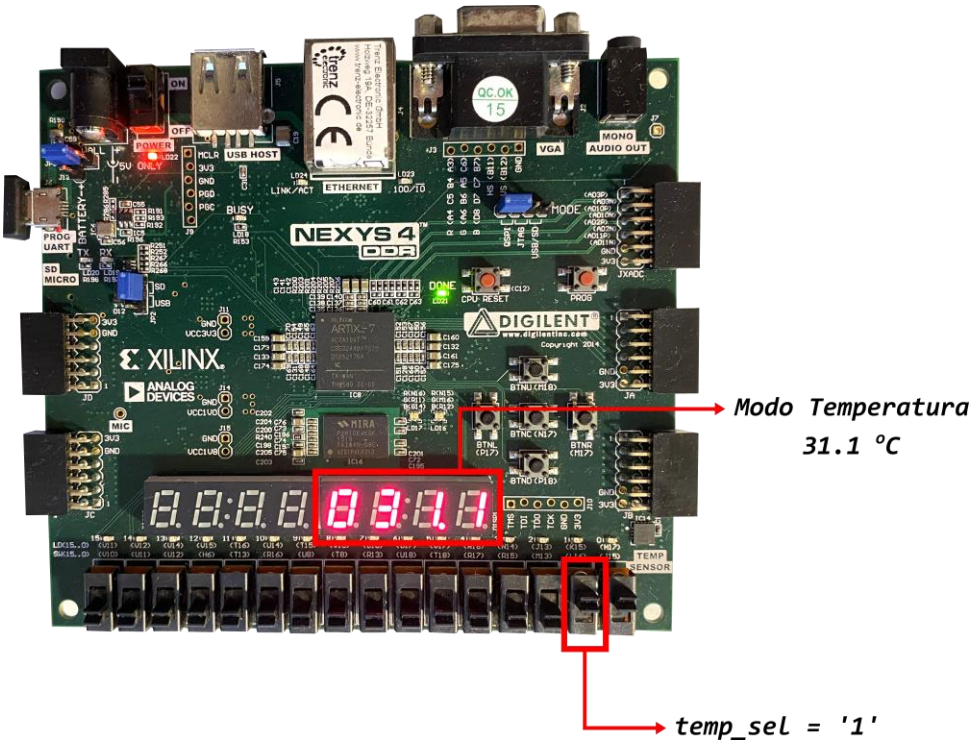


Fig.40. Imagen real del Modo Temperatura del sistema de visualización multiplexada de las señales.

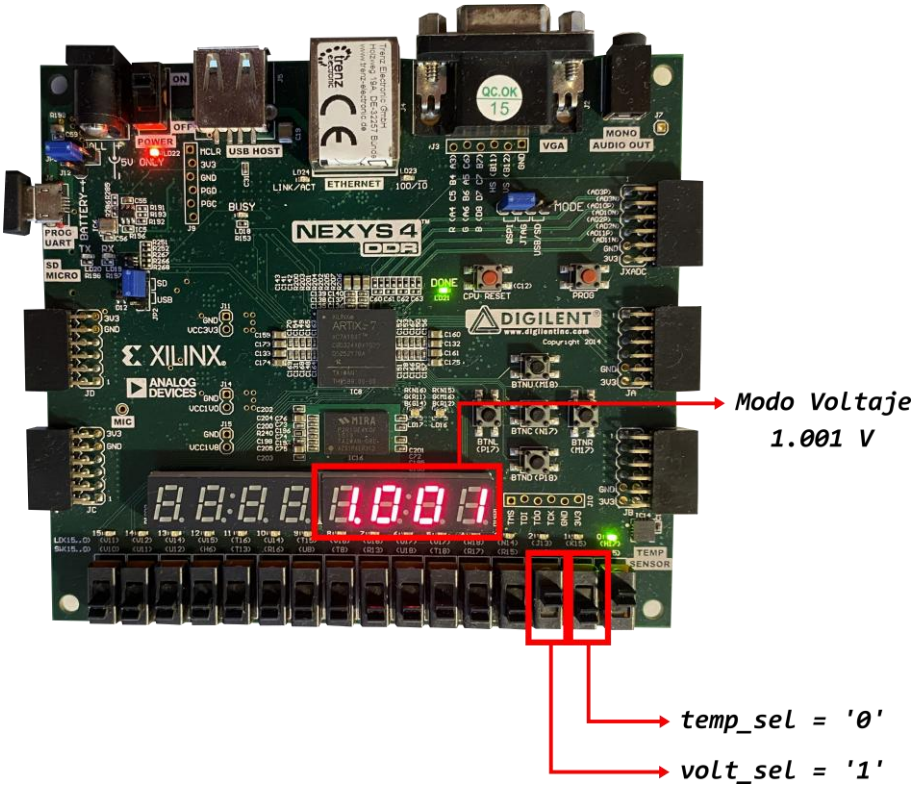


Fig.41. Imagen real del Modo Voltaje del sistema de visualización multiplexada de las señales.